코드 검색·RAPTOR 요약·파일 전체 입력 비교 보고서

이 실험은 **AI 언어 모델(LLM)에게 코드의 동작을 물을 때, 어떤 자료를 찾아 보내야 적은 입력으로 정확하게 답하는가**를 비교한다. 같은 답변 모델과 질문을 사용하면서, 함께 전달할 코드를 고르는 방법과 요약 사용 여부를 바꿨다. 현재 색인(A)은 필요한 코드만 찾아 보내 컨텍스트 사용을 줄이려는 방식이며, 나머지 방식은 찾는 범위와 전달 자료를 달리한 비교 대상이다.

토큰은 모델이 글과 코드를 처리하는 길이 단위다. **컨텍스트 윈도우**는 한 번의 모델 호출에서 입력과 생성할 출력을 함께 담을 수 있는 한도다. 파일을 통째로 보내면 선택 과정이 단순하지만 관계없는 코드도 입력을 차지한다. 필요한 부분만 고르면 입력을 줄일 수 있는 대신, 중요한 규칙까지 빠질 수 있다.

비교 기준인 '파일 전체'는 대상 함수가 정의된 파일 하나 전체다. 그 파일의 경로는 미리 알려줬다. 프로젝트 전체나 관련 파일 전부를 넣는 방식은 실험하지 않았다. 원문의 `import`문만 보낸다고 다른 파일의 본문까지 이번 입력에 들어가는 것은 아니므로, 여러 파일에서 필요한 부분을 모은 방식이 파일 하나 전체보다 더 완전한 답변 근거를 줄 수도 있다.

정답률은 최종 답변이 맞은 비율이다. 반환값·조건·호출 함수 목록 등에 대한 기준 답을 정확한 JSON 값으로 출력했는지 평가했다. 필요한 코드를 찾는 데 성공해도 모델이 계산이나 답변 형식을 틀리면 오답이다. 검색 결과의 적중률과 구분해야 하며, 보안 취약점 탐지 성능을 측정한 것은 아니다.

기존 네 방식 실험: **2026년 10월 24일** · 큰 파일 RF·RH·C 추가 결과 집계: **2026년 10월 25일** · 평가 범위: **합성 Python 프로젝트 6개, 서로 다른 96문제, 한 모델**.

파일 하나 전체와 비교해 얻은 것과 남은 대가

- **큰 파일에서 입력 절약:** 현재 색인(A)의 질문당 총 입력은 500.9토큰으로 파일 전체의 4265.8토큰보다 **88.3% 적었다.** 질문마다 가장 큰 호출의 입력을 고른 값의 평균도 342.0토큰으로, 파일 전체보다 92.0% 작았다. 정답은 A 17개, 파일 전체 15개였다.
- **큰 파일에서 관측 정답 수:** 원문 검색(R0)은 총 입력을 **72.7% 줄이면서 25/96개**를 맞혔다. 파일 전체는 15/96개였다. 다만 R0도 정답률이 26.0%에 그쳐 충분한 신뢰성을 확보한 결과는 아니다.
- **요약을 추가한 결과:** 함수 요약(RF) 15개, 계층 요약(RH) 16개, 계층 요약 정리(C) 16개로, **R0보다 더 정확해지지는 않았다.** C는 RH의 같은 고유 자료를 더 짧게 전달해 총 입력을 18.2% 줄였지만, 개별 문제에서는 2개 개선·2개 악화가 있었다. 요약을 미리 만드는 비용도 별도로 든다.
- **작은 파일에서는 다른 결과:** 파일 전체의 질문당 입력은 316.7토큰, A는 409.1토큰이었고 정답은 둘 다 17개였다. 파일이 작으면 선택 호출의 비용 때문에 일부를 골라도 누적 입력이 더 클 수 있다.

이 수치는 이번 표본의 관측 결과다. **입력 절감과 정확도 개선을 따로 판단해야 하며**, 통계적 불확실성·실행 조건·사전 준비 비용은 본문에서 함께 다룬다.

읽는 순서: 결과를 먼저 보려면 4절 → 5절 → 7절을 읽으면 된다. 방식과 배경부터 이해하려면 1절부터 순서대로 읽는다.

본문	세부 자료
1. 일곱 방식의 원리와 실제 색인 예시	부록 A. 상세 성적과 통계
2. UI에서 색인을 만드는 주체와 과정	
3. 실험 조건과 지표를 읽는 법	부록 B. 같은 질문의 실제 답변
4. 정답률·시간·파일 크기에 따른 결과	부록 C. 원자료·검증·재현
5. 토큰·컨텍스트·준비 비용	

본문	세부 자료
6. 왜 이런 정답률이 나왔는가	
7. 목적에 따른 판단과 한계	

1. 일곱 방식의 원리와 실제 색인 예시

색인은 **질문을 받았을 때 어떤 코드를 꺼낼지 찾기 위해 미리 준비하는 자료**다. 색인에 저장한 내용 전체가 LLM(질문을 읽고 답하는 언어 모델)에 전달되는 것은 아니다. 아래에서는 **원본 파일 → 저장하는 자료 → 질문이 들어왔을 때 꺼내는 코드** 순서로 살펴본다.

1.1 파싱, 색인, LLM 요약의 차이

색인을 쓰는 방식의 전체 과정부터 보면, 먼저 프로젝트 원문에서 찾기용 자료를 준비한다. 질문이 들어오면 프로그램이 그 자료를 이용해 원문 조각이나 요약을 고르고, **질문과 선택 자료를 함께 LLM에 보내 답을 받는다.** 색인 전체를 매번 보내는 것이 아니다. RF·RH의 요약은 질문 전에 LLM으로 만들고, C는 그 요약을 재사용한다.

함수는 입력을 받아 계산하거나 작업하는 코드 단위이고, 상수는 코드에서 정해 둔 값이다. 한 함수가 다른 파일의 함수를 호출하거나 상수를 사용할 수 있어, 답에 필요한 코드가 여러 파일에 나뉘기도 한다.

각 비교는 다음 질문을 확인하기 위해 두었다. 아래 각 행은 별도로 실행한 두 방식의 비교를 뜻한다.

비교	확인하려는 질문
파일 전체 ↔ 현재 색인(A)	파일 하나를 모두 보내는 대신 주소록으로 필요한 코드만 찾으면 입력을 얼마나 줄이고, 어떤 근거를 놓치는가?
현재 색인(A) ↔ 파일 간 확장(B)	같은 출발 함수에서 다른 파일의 참조까지 따라가면 부족한 계산 규칙을 보충할 수 있는가?
A·B ↔ 원문 검색(R0)	출발 함수 하나를 LLM이 고르는 대신, 질문의 단어로 여러 원문 후보를 검색하면 무엇이 달라지는가?
원문 검색(R0) ↔ 함수 요약(RF)	코드의 역할을 설명한 문장을 검색 후보에 더하면 관련 내용을 더 잘 찾고 답할 수 있는가?
함수 요약(RF) ↔ 계층 요약(RH)	여러 함수의 설명을 파일·프로젝트 수준으로 묶으면 관련 근거를 찾고 정확한 답을 내는 데 도움이 되는가?
계층 요약(RH) ↔ 계층 요약 정리(C)	같은 자료에서 반복과 긴 출처 표기를 줄이면 입력·시간·답변이 어떻게 달라지는가?

이제 준비 과정에서 **코드 구조를 읽는 작업**과 **새 설명을 생성하는 작업**을 구분한다.

파싱은 소스 코드를 문법 구조로 읽는 작업이다. 여기서는 Python의 `ast.parse`로 **AST(함수·변수·import문 등의 구조를 표현한 문법 트리)**를 만든다. 예를 들어 함수 이름과 시작 줄, 참조한 이름을 읽을 수 있으며, 이 단계에서는 코드를 실행해 결과를 계산하지 않는다. **자연어 요약**은 그 코드가 무엇을 하는지 설명하는 새 문장을 만드는 작업이다. 아래에서 두 작업을 구분한다.

A·B·R0의 준비 자료는 파싱·규칙 계산으로 만들고, RF·RH의 설명 문장은 LLM이 만든다. C는 RH의 LLM 요약을 재사용하고 전달 형식만 코드로 바꾼다. 모든 방식의 최종 답변은 LLM이 생성한다. A·B의 '함수를 고르는 LLM 호출'도 사전 요약 생성과 구분해야 한다.

1.2 모든 방식에 사용할 같은 원본 파일

설명을 위해 아래 세 파일을 만들었다. 실제 실험 구현으로 색인과 검색 후보를 생성했으며, 이 작은 예제는 뒤의 평가용 96문제와 별개다.

`service.py` — 잔여 수량을 묻는 함수와, 질문에 관계없는 표시 함수가 함께 있다.

```
from rules import sellable

def remaining(on_hand, held):
    return sellable(on_hand, held)

def label(on_hand):
    return 'empty' if on_hand <= 0 else 'stocked'
```

`rules.py` — 실제 수량 계산은 다른 파일에 있다.

```
from policy import SAFETY_STOCK

def sellable(on_hand, held):
    return max(0, on_hand - held - SAFETY_STOCK)
```

`policy.py` — 계산에 필요한 안전재고 값이다.

```
SAFETY_STOCK = 3
```

질문은 **"`remaining(9, 1)`의 반환값은?"**이다. 계산식은 `max(0, 9 - 1 - 3)`이므로 정답은 `5`다. 아래 A·B 설명에서는 AI가 출발 함수로 `remaining`을 골랐다고 가정한다. 이 예제에서는 질문에 대한 LLM 답변은 생성하지 않았다. 요약과 검색 자료는 실제 구현으로 생성했으며, 정답률은 4절의 평가에서 측정한다.

먼저 파일 전체 방식으로 보면, `service.py`의 모든 줄을 보내므로 관계없는 `label`까지 포함하지만, 실제 계산식이 있는 `rules.py`와 값 `3`이 있는 `policy.py`는 빠진다. B는 세 파일에서 계산에 필요한 조각만 모은다. 이 예제는 **파일 하나의 모든 줄을 보내는 것과 답에 필요한 규칙을 모두 보내는 것이 다름**을 보여 준다.

이제 같은 `remaining(9, 1)` 질문에 대해, 각 방식이 **무엇을 저장하고 무엇을 전달하는지** 비교한다.

1.3 A — 현재 색인: 이름과 위치로 필요한 원문을 찾는다

단계	처리 내용
질문 전 준비	파일 파싱 → 함수·클래스의 이름, 파일, 줄 위치를 추출 → `symbol_index` 주소록 저장
사전 LLM 요약	**없음.** 이름·위치는 파서가 추출하며 요약 문장은 만들지 않음

단계	처리 내용
질문 후 처리	이름·경로에 관한 규칙으로 후보를 최대 24개 추림 → LLM이 출발 함수 선택 → 파서로 같은 파일의 관련 원문을 모음 → LLM 답변

A의 `symbol_index`는 **코드 본문을 복사한 저장소가 아니라 함수 등의 위치를 적은 주소록**이다. `remaining`에 해당하는 실제 항목은 다음과 같다.

```
{
  "path": "service.py",
  "kind": "function",
  "name": "remaining",
  "qualname": "remaining",
  "start_line": 3,
  "end_line": 4,
  "uri": "memory://symbol/service.py::remaining"
}
```

`path`는 파일, `start_line`·`end_line`은 그 함수가 있는 줄, `uri`는 함수를 구분하는 주소다. 여기에 `return sellable(...)` 같은 **함수 본문은 저장하지 않는다.** `label`과 `rules.py`의 `sellable`도 각각 같은 형식의 항목을 가진다. `SAFETY_STOCK` 같은 상수는 이 기본 함수 색인에 들어가지 않고, 원문을 읽으며 같은 파일의 참조 상수를 모으는 단계에서 처리한다.

질문이 오면 후보 함수들의 이름·위치를 LLM에 보여 주어 하나를 고르게 한다. `remaining`을 고르면 `service.py`의 3~4줄을 읽고, 같은 파일에서 연결된 함수·상수를 찾는다. 이 예제에서 답변용으로 꺼내는 코드 본문은 아래 두 줄이다. 실제 입력에는 식별용 제목과 코드 표시도 붙는다.

```
def remaining(on_hand, held):
    return sellable(on_hand, held)
```

질문에 무관한 `label`은 제외된다. `sellable`은 색인에 위치가 있어도 **다른 파일에 있으므로 A의 확장 규칙으로 따라가지 않는다.** 이 예제에서는 실제 계산식과 안전재고 값이 답변 자료에 들어가지 않는다. A가 파일 전체보다 적은 컨텍스트를 쓰는 이유와, 그 과정에서 근거가 빠질 수 있는 이유를 함께 보여 준다.

1.4 B — 파일 간 확장: 다른 파일의 참조까지 따라간다

단계	처리 내용
질문 전 준비	A의 주소록 + 파싱한 함수·상수·import 원문 → 참조 이름과 `from … import …` 연결을 코드로 계산 → 파일 간 참조 관계 저장
사전 LLM 요약	**없음.** 참조 관계도 LLM이 추측해 만드는 것이 아님
질문 후 처리	A와 같은 후보·선택 절차 → 참조 관계를 2차례 따라 원문 확장(함수 최대 8개) → LLM 답변

B도 출발 함수를 고를 때 A와 같은 `symbol_index`를 사용한다. 여기에 코드 분석으로 만든 `nodes` 자료를 추가한다. 다음은 **실제 생성한 객체의 핵심 필드만 발췌**한 것이다.

```
{
  "memory://symbol/service.py::remaining": {
    "source": "def remaining(on_hand, held):\n    return sellable(on_hand,
held)",
    "imports": "from rules import sellable",
    "deps": [
      "memory://symbol/rules.py::sellable"
    ]
  },
  "memory://symbol/rules.py::sellable": {
    "source": "def sellable(on_hand, held):\n    return max(0, on_hand -
held - SAFETY_STOCK)",
    "imports": "from policy import SAFETY_STOCK",
    "deps": [
      "memory://symbol/policy.py::SAFETY_STOCK"
    ]
  },
  "memory://symbol/policy.py::SAFETY_STOCK": {
    "source": "SAFETY_STOCK = 3",
    "imports": "",
    "deps": []
  }
}
```


`source`에는 해당 함수·상수의 원문, `imports`에는 그 파일의 import문, `deps`에는 참조 대상의 주소가 들어 있다. 즉 B는 주소만 더 갖는 것이 아니라 **원문과 파일 간 연결 정보도 사전에 준비**한다. `remaining → sellable → SAFETY_STOCK`이라는 연결을 프로그램이 따라갈 수 있게 된다.

질문이 오면 `remaining`에서 시작해 두 차례 확장하여 세 코드의 원문을 수집한다. `service.py`나 `rules.py`를 통째로 가져오는 것이 아니므로, 이 예제에서는 무관한 `label`을 넣지 않는다. 실제 입력에는 각 조각의 제목과 import문도 붙는다.

1.5 R0 — 원문 검색: 원문 조각과 참조 묶음을 검색한다

단계	처리 내용
질문 전 준비	B와 같은 정적 참조 자료 → 함수·상수 원문을 최대 3단계 참조와 묶고 단독 원문 후보도 준비 → 단어 가중치 계산
사전 LLM 요약	**없음.** 원문 조각을 복사·묶으며 자연어로 바꾸지 않음
질문 후 처리	TF-IDF 단어 점수로 원문 후보 정렬 → 1,024토큰 안에서 선택 → LLM 답변

여기서 **'후보'는 질문에 답할 근거로 LLM에게 보낼지 판단하기 위해, 검색 목록에 미리 등록해 둔 텍스트 항목 하나**를 뜻한다. 예를 들어 `remaining` 함수의 원문만 담은 항목이 후보 하나이고, `remaining` + `sellable` + `SAFETY_STOCK`의 원문을 함께 담은 항목이 또 다른 후보다. 두 항목에 일부 코드가 겹쳐도 검색에서는 별개 후보로 다룬다. 프로젝트를 색인할 때 이런 목록을 준비하고, 질문이 들어오면 각 후보에 점수를 매겨 일부를 고른다. **'후보'라는 말은 아직 최종적으로 LLM에 보낼 자료로 선택되기 전이라는 뜻**이다.

TF-IDF는 질문과 코드에 공통으로 등장하는 단어의 중요도를 계산하는 방법이다. TF(Term Frequency, 단어 빈도)는 '이 후보 안에서 몇 번 나오는가', IDF(Inverse Document Frequency, 역문서 빈도)는 '전체 후보 중 얼마나 적은 후보에 나오는가'를 뜻한다. 여기서 **문서는 파일 하나가 아니라 검색 후보 하나**다. 함수 단독 원문도, 여러 함수·상수를 묶은 원문도 각각 한 후보로 센다. 이 실험은 TF-IDF로 단어별 가중치를 만든 다음, 질문과 후보의 가중치 구성이 얼마나 비슷한지를 검색 점수로 사용한다.

따라서 TF의 **'이 후보'**는 텍스트 항목 하나를, IDF의 **'전체 후보'**는 현재 프로젝트에서 준비한 원문 텍스트 항목의 전체 목록을 가리킨다. 예를 들어 위 두 항목에 `remaining`이 모두 들어 있으면, 그 단어는 서로 다른 후보 **2개**에 등장한 것으로 센다.

색인 준비와 질문 처리는 서로 다른 시점에 실행한다. '색인 생성' 버튼의 입력은 프로젝트 코드이며, 이때 사용자의 질문은 필요 없다. 아래 설명에는 준비한 색인을 실제로 사용하는 검색 과정도 포함되어 있다.

시점	입력	프로그램이 하는 일
색인·검색 자료 준비	현재 프로젝트의 코드	원문 후보 생성 → 후보별 단어 추출 → 원문 후보 전체의 IDF 계산 → 후보별 TF-IDF와 정규화된 벡터 준비
사용자 질문 접수 후 검색	사용자가 입력한 질문 문자열	질문의 단어 추출 → 질문 안의 반복 횟수 계산 → **기존 IDF** 적용·정규화 → 후보와 점수 비교 → 자료 선택 → 원래 질문과 선택 자료로 LLM 답변

실제 실험에서는 저장된 자료를 읽은 뒤 평가 시작 시 후보 벡터를 메모리에 준비했다. UI 버튼으로 연결할 때도 질문 없이 준비할 수 있으며, 파일 저장과 메모리 준비의 차이는 2.4절에 설명한다. **새 질문이 들어왔다고 원문 후보의 수나 IDF를 다시 계산하지는 않는다.**

① 후보에서 비교할 단어를 추출한다. 질문에도 검색 시 같은 규칙을 쓴다. 검색 코드가 영문을 소문자로 바꾸고 식별자를 나눈다. 예를 들어 `SAFETY_STOCK`에서는 `safety_stock`, `safety`, `stock`을, `remainingStock`에서는 `remaining`, `stock`을 얻는다. 실제 후보의 `text`에는 원문뿐 아니라 경로·제목·import문도 있으므로 그 안의 단어도 계산에 들어간다. 이 구현은 영문·숫자 중심으로 추출하고 한 글자짜리는 제외한다. 따라서 한글만 있는 질문에서는 검색어가 남지 않을 수 있다. 여기서 세는 '단어'는 LLM의 입력 길이를 재는 '토큰'과 다르다.

사용자의 질문은 어떻게 검색용 단어가 되는가? — 질문을 받은 뒤 수행하는 처리

UI에서 받은 질문 문자열을 백엔드가 검색 코드의 `words(question)`에 전달하는 방식으로 연결한다. 현재 실험은 UI 대신 평가 문제의 질문 문자열을 같은 코드에 전달했다. 원문 코드의 함수·상수 구조를 읽는 `ast.parse`와 달리, **질문에는 문법 트리를 만들지 않고 정해진 문자 규칙으로 단어를 분리**한다. 형태소 분석, 번역, 질문 의도 파악이나 LLM을 통한 핵심어 추출은 이 구현에 없다.

순서	질문을 처리하는 규칙	예시
1. 식별자 분리·소문자화	영문 소문자 뒤에 대문자가 오는 경계를 나누고 소문자로 바꾼다.	`remainingStock` → `remaining` `stock`
2. 단어 추출	영문자로 시작하는 영문·숫자·밑줄 묶음, 또는 연속된 숫자를 찾는다. 공백·문장부호는 단어에 포함하지 않는다.	`Stock hold?` → `stock`, `hold`
3. 밑줄 단어 확장·짧은 항목 제외	밑줄 식별자는 원형과 분리한 부분을 함께 넣고, 길이가 한 글자인 항목은 제외한다.	`SAFETY_STOCK` → `safety_stock`, `safety`, `stock`; `x`, `3`은 제외, `10`은 유지

순서	질문을 처리하는 규칙	예시
4. 질문의 TF-IDF 계산	추출 목록의 반복 횟수를 세고, 후보에서 미리 구한 IDF를 곱한 뒤 정규화한다.	`stock`이 두 번 추출되면 질문에서도 TF는 `1 + ln(2)`
5. 검색과 답변	질문 벡터와 후보 벡터를 비교해 자료를 고른다. LLM에는 **사용자가 입력한 원래 질문 전체**와 선택 자료를 전달한다.	검색용 단어 목록으로 원래 질문을 대체하지 않음

다음은 실제 `words()`로 확인한 결과다.

사용자가 입력했다고 가정한 문자열	추출된 검색용 단어 목록
`Stock hold?` — 계산 설명용 짧은 검색어	`stock`, `hold`
`How does remainingStock use SAFETY_STOCK?`	`how`, `does`, `remaining`, `stock`, `use`, `safety_stock`, `safety`, `stock`
`재고가 얼마나 남아 있나요?`	없음: `[]`

둘째 예제에서는 식별자를 나누면서 `stock`이 두 번 생긴다. 또 `how`, `does`, `use` 같은 일반 단어도 남는다. 즉 **중요한 단어를 의미로 판단해 골라내는 것이 아니라, 규칙에 맞는 단어를 모두 추출하고 가중치로 비교**한다.

한글만 있는 마지막 예제는 현재 규칙으로 단어가 추출되지 않아 모든 후보의 검색 점수가 0이 된다. 현재 코드는 이때 검색 실패를 별도로 반환하지 않고 후보 ID 순으로 정렬하므로, 관련 자료를 제대로 찾았다고 볼 수 없다. 한국어 질문을 지원하는 UI를 만들려면 한국어 단어 추출과 검색 대응 방식, 검색어가 비었을 때의 처리를 별도로 설계해야 한다. 이는 현재 구현된 기능이 아니다.

이하의 `stock hold`는 첫 행에서 추출된 두 단어를 사용한 **검색 시점의 계산 예제**다. 색인 생성 버튼에 질문을 함께 입력하라는 뜻이 아니다. 후보의 TF-IDF 계산은 질문 전에 준비하고, 질문의 TF-IDF 계산과 후보 비교는 질문을 받은 뒤 수행한다.

② 단어가 몇 개 후보에 등장하는지 세어 희소성(IDF)을 구한다. 전체 원문 후보 수를 `N`, 단어 `w`가 한 번 이상 등장한 후보 수를 `df(w)`라고 하면 실제 코드의 식은 다음과 같다.

```
IDF(w) = ln((N + 1) / (df(w) + 1)) + 1
```

`ln`은 자연로그다. 비율이 커질수록 값은 커지지만 증가 폭은 완만해진다. 분자·분모의 `+1`은 등장 후보가 0개인 단어도 계산할 수 있게 하고, 마지막 `+1`은 모든 후보에 등장한 단어도 가중치 1을 갖게 한다. **한 후보 안에 열 번 나와도 `df`에는 1만 더한다.** 같은 코드가 여러 참조 묶음에 들어 있다면 각 묶음은 별도 후보이므로 각각 센다.

계산만 이해하기 위해 후보를 `stock stock hold`, `stock price`, `price`의 세 개로 줄여 보자. 아래 수치는 **실제 검색 코드로 계산한 설명용 예제**이며, 앞의 세 파일이나 평가 결과를 대신하는 실험은 아니다.

단어	등장한 후보 수 `df`	희소성 계산 (`N = 3`)	IDF
`stock`	2	`ln(4 / 3) + 1`	1.288
`hold`	1	`ln(4 / 2) + 1`	1.693
`price`	2	`ln(4 / 3) + 1`	1.288

`hold`는 한 후보에만 있으므로 두 후보에 있는 `stock`보다 더 큰 가중치를 받는다. 이것이 이 코드에서 말하는 **단어의 희소성**이다.

③ 각 후보 안의 반복 횟수(TF)와 희소성(IDF)을 곱한다. 단어가 그 후보 안에서 `c`번 등장하면, 실제 코드는 등장한 단어에 대해 다음처럼 계산한다. 없는 단어의 가중치는 0이다.

`TF = 1 + ln(c)` → `단어 가중치 = TF × IDF`

한 번 나오면 TF는 1, 두 번 나오면 약 1.693이다. 반복 횟수를 그대로 쓰지 않아 같은 단어의 반복이 점수를 지나치게 키우는 것을 줄인다. 예를 들어 첫 후보의 `stock`은 두 번 나오므로 `1.693 × 1.288 = 2.180`, `hold`는 한 번이므로 `1 × 1.693 = 1.693`이 된다. **검색 시점에 질문이 들어오면 질문에도 같은 식과 기존 IDF를 적용**한다. 예를 들어 사용자가 `Stock hold?`를 입력하면 `stock`, `hold`를 추출하고, 이 설명용 후보 세 개에서 구한 IDF를 적용해 질문 가중치 1.288과 1.693을 얻는다.

④ 후보의 길이를 보정하고, 질문이 들어오면 질문도 같은 방식으로 보정해 검색 점수를 구한다. 단어별 가중치를 모은 수치 목록을 '벡터'라고 부른다. 각 가중치를 `√(모든 가중치의 제곱 합)`으로 나누어 벡터 길이를 1로 맞춘다. 이를 **정규화**라고 하며, 단어 수가 많은 후보가 단순히 합계만 커서 유리해지는 것을 줄인다. 질문 `stock hold`의 정규화된 가중치는 `stock: 0.605`, `hold: 0.796`이다.

검색 점수는 **질문과 후보에서 같은 단어의 정규화된 가중치를 곱한 값의 합**이다. 이 계산을 **코사인 유사도**라고 한다.

원문 후보	정규화 전 가중치	정규화 후 가중치	질문 `stock hold`와의 점수
`stock stock hold`	`stock: 2.180`, `hold: 1.693`	`stock: 0.790`, `hold: 0.613`	**0.966**
`stock price`	`stock: 1.288`, `price: 1.288`	`stock: 0.707`, `price: 0.707`	**0.428**
`price`	`price: 1.288`	`price: 1.000`	**0.000**

첫 후보는 `0.605 × 0.790 + 0.796 × 0.613 ≈ 0.966`이다. 둘째는 공통 단어가 `stock`뿐이므로 `0.605 × 0.707 ≈ 0.428`, 셋째는 공통 단어가 없어 0이다. 표는 소수 셋째 자리로 반올림했고 실제 계산에는 반올림 전 값을 사용했다. 이 점수는 이 구현에서 0~1 범위이며, **정답일 확률이 아니라 단어 구성의 유사도**다.

⑤ 높은 점수부터 원문을 골라 LLM에 보낸다. R0은 이 점수로 후보를 정렬하고, 이미 선택한 원문에 완전히 포함되는 후보는 건너뛴다. 남은 후보를 합쳤을 때 1,024토큰 한도에 들어오면 추가하고, 넘으면 그 후보 전체를 건너뛰어 다음 후보를 살핀다. LLM에는 이렇게 선택된 **원문 텍스트**를 보내며, TF-IDF 가중치 표 자체를 답변 자료로 보내는 것은 아니다.

RF·RH·C에도 같은 검색 계산을 쓴다. 단, 희소성의 기준인 `N`과 `df`는 프로젝트의 **원문 후보만으로 계산**하고, 그 IDF를 원문·요약·질문 모두에 적용한다. 요약을 추가했다고 IDF를 다시 계산하지 않는다. 원문 후보에 없던 단어는 `df = 0`으로 취급하므로 위 예제에서는 `ln(4) + 1 ≈ 2.386`의 가중치를 받는다. 따라서 요약에만 생긴 표현도 큰 가중치를 받을 수 있지만, 그것이 답에 필요한 정보라는 뜻은 아니다.

이 과정은 단어 분리와 TF-IDF 검색 코드가 계산한다. R0·RF·RH·C의 검색 단계에는 임베딩 모델이나 LLM을 이용한 후보 재평가를 사용하지 않았다. 토큰 길이를 세는 작업도 요약 문장을 생성하는 작업과 별개다.

R0의 끝 글자는 숫자 `0`이다. R0은 LLM에게 출발 함수 하나를 고르게 하지 않는다. B와 같은 종류의 참조 정보를 이용해 **질문이 오기 전에 원문 후보를 만들고**, 질문과 후보에 등장하는 단어를 비교해 검색한다.

`remaining`에서는 다음 두 후보가 만들어진다. 아래는 실제 후보 객체에서 `id`와 `source_uris`만 발췌한 것이다. `source_uris`는 그 후보에 어떤 원문이 담겼는지 나타낸다.

```
[
  {
    "id": "atomic:memory://symbol/service.py::remaining",
    "source_uris": [
      "memory://symbol/service.py::remaining"
    ]
  },
  {
    "id": "memory://symbol/service.py::remaining",
    "source_uris": [
      "memory://symbol/service.py::remaining",
      "memory://symbol/rules.py::sellable",
      "memory://symbol/policy.py::SAFETY_STOCK"
    ]
  }
]
```


첫 후보는 `remaining` 하나의 원문이고, 두 번째는 관련 함수·상수까지 묶은 원문이다. **각 후보에는 `text` 필드로 실제 원문도 저장된다.** 예를 들어 첫 후보의 `text`는 다음과 같다.

```
### memory://symbol/service.py::remaining lines 3-4
from rules import sellable
def remaining(on_hand, held):
    return sellable(on_hand, held)
```

두 번째 후보의 `text`에는 이 내용에 `sellable`과 `SAFETY_STOCK`의 원문이 이어 붙는다. 실제 구현은 참조 연결을 최대 3단계 따라 묶으며, `label` 등 다른 코드의 후보도 함께 만든다. 전체 프로젝트를 하나의 큰 파일로 저장하는 방식은 아니다.

또한 검색 시작 전에 후보마다 단어의 가중치를 계산해 **메모리에 보관**한다. `remaining` 단일 후보의 계산 결과 중 일부는 아래와 같다. 소수 넷째 자리로 반올림했다.


```
{
  "remaining": 0.4755,
  "sellable": 0.2971,
  "held": 0.344
}
```


이 값은 코드에 나타난 단어의 빈도와 희소성으로 계산한 **TF-IDF 가중치**다. 의미를 학습한 임베딩 모델의 출력은 아니다. 질문에도 같은 계산을 적용해 단어 구성이 비슷한 후보부터 정렬한다. 이 예제에서는 `remaining` 단일 원문이 먼저, 관련 코드 묶음이 다음으로 검색됐다.

검색 결과는 높은 순서대로 1,024토큰 안에 담는다. 이미 들어간 코드만 반복하는 후보는 건너뛰지만, 일부만 겹치는 묶음은 코드가 중복될 수 있다. 최소 점수 기준이 없어 공간이 남으면 관련성이 낮은 후보도 들어갈 수 있다. 따라서 **R0도 파일 전체 대신 원문을 고르지만, 항상 가장 적은 양만 보내는 방식은 아니다.**

1.6 RF — 함수 요약: 원문 후보에 LLM 설명을 추가한다

왜 요약을 시도했나? 질문이 함수 이름 대신 기능을 설명하는 말로 주어지면, 코드의 역할을 풀어 쓴 문장이 관련 자료를 찾는 데 도움이 될 수 있다는 기대 때문이다. RH는 여기에 여러 함수의 관계와 흐름을 짧게 묶으려는 아이디어를 더한다. 이는 **기대한 이점**이며, 이번 구현에서 실제로 좋아졌다는 결론은 아니다.

이번 검색은 요약을 넣어도 단어 점수를 사용한다. 또 평가는 수치·분기·호출 관계를 정확히 답해야 하는 문제이므로, 설명이 읽기 쉬워져도 필요한 상수나 조건이 빠지면 불리할 수 있다. 아래 예제의 실제 요약에서는 그 차이를 확인할 수 있다.

단계	처리 내용
질문 전 준비	R0의 원문 후보 → 각 함수와 연결된 원문 묶음을 LLM에 전달 → 함수 요약을 원문과 함께 저장
사전 LLM 요약	**있음.** 평가용 6개 프로젝트에서 함수 요약 1,848개를 실제 생성
질문 후 처리	원문과 함수 요약을 같은 단어 점수로 정렬 → 1,024토큰 안에서 선택 → LLM 답변

RF·RH의 공통 요약 지시와 처리 방식

RF·RH의 요약에는 답변과 같은 **Qwen2.5-Coder-7B-Instruct Q5_K_M**을 사용했다. RF에는 '동작·정확한 수식·분기·문자열 규칙·호출·상수를 보존하라', RH에는 '하위 요약의 흐름·관계·예외와 제공된 수치·조건을 보존하라'고 지시했다. 두 단계 모두 **영어 100단어 이내를 요청하고 출력은 최대 256토큰**으로 제한했으며, 질문과 정답은 입력하지 않았다. 이는 생성 지시이지, 모든 세부 정보가 보존됐다는 검증 결과는 아니다.

RH의 파일·프로젝트 요약은 **하위 요약을 입력으로 다시 생성**한다. 그 단계에서 원문을 다시 대조해 검증하지는 않는다. C의 중복 제거는 저장된 식별자와 본문을 사용하는 일반 프로그램 처리이며, LLM이 내용을 다시 쓰지 않는다. 아래 실제 저장 예제에서 이 과정을 확인할 수 있다.

RF는 **함수 요약 방식**이다. R0이 준비한 원문 후보를 그대로 두고, 각 함수와 연결된 코드 묶음을 LLM으로 읽어 만든 요약을 추가한다. 위 예제의 `remaining` 관련 코드 묶음을 요약하면, 해당 함수뿐 아니라 `sellable`과 `SAFETY_STOCK`도 요약의 재료가 된다. 별도로 저장한 함수 단독 원문 후보에는 중복 요약을 만들지 않는다. 질문을 받기 전에 생성하고 저장한다.

실제로 생성한 요약 항목의 연결 정보는 다음과 같다. `level: 1`은 함수 단계, `children`은 직접 요약한 원문 후보, `source_uris`는 그 후보의 출처를 뜻한다.


```
{
  "id": "summary:leaf:memory://symbol/service.py::remaining",
  "level": 1,
  "children": [
    "memory://symbol/service.py::remaining"
  ],
  "source_uris": [
    "memory://symbol/service.py::remaining",
    "memory://symbol/rules.py::sellable",
    "memory://symbol/policy.py::SAFETY_STOCK"
  ]
}
```


이 항목의 `text`에는 출처 제목과 다음 **실제 LLM 요약 본문**이 저장되었다. 아래는 읽기 쉽게 줄만 나눴다.

```
The `remaining` function calculates the number of items that can be sold by
subtracting the number of items held from the number of items on hand and
```

```
subtracting a safety stock. It uses the `sellable` function from the
`rules`
module, which in turn uses the `SAFETY_STOCK` constant from the `policy`
module. The `sellable` function ensures the result is not negative by using
the `max` function.
```

이 요약은 '보유 수량에서 예약 수량과 안전재고를 빼고, 음수가 되지 않게 한다'는 설명이다. 출처 목록에는 SAFETY_STOCK이 있지만 위 함수 요약 본문에는 **숫자 3이 빠져 있다.** 출처가 연결되어 있다는 것과 세부 값이 남아 있다는 것은 다르다.

질문이 오면 **원문 후보와 함수 요약 후보를 한 목록에 놓고** R0과 같은 단어 유사도로 순위를 정해 1,024토큰 안에 넣는다. 함수 주소만 전달하거나 요약만 전달하는 방식이 아니다. 요약이 원문보다 짧을 수는 있지만, 원문과 요약을 함께 선택할 수 있어 전체 입력이 반드시 줄지는 않는다. 또한 '출처에 포함되었다'는 사실만으로 요약에 그 출처의 모든 규칙이 남아 있다고 볼 수 없다.

1.7 RH — 계층 요약: 함수 설명을 파일·프로젝트로 재요약한다

단계	처리 내용
질문 전 준비	RF의 함수 요약 → 같은 파일의 요약들을 LLM으로 재요약 → 파일 요약들을 LLM으로 재요약해 프로젝트 요약 저장
사전 LLM 요약	**있음.** 함수 1,848개 + 파일 30개 + 프로젝트 6개. 내용은 LLM이 쓰고, 파일별 묶음은 코드가 정함
질문 후 처리	원문·함수·파일·프로젝트 요약을 한 목록에서 검색 → 일부 선택 → LLM 답변

RH는 **계층 요약 방식**이다. RF의 함수 요약들을 파일별로 묶어 다시 요약하고, 파일 요약들을 모아 프로젝트 요약을 만든다. 저장하는 연결 구조의 일부는 다음과 같다.

```
[
  {
    "id": "summary:level2:service.py",
    "level": 2,
    "children": [
      "summary:leaf:memory://symbol/service.py::remaining",
      "summary:leaf:memory://symbol/service.py::label"
    ]
  },
  {
    "id": "summary:level3:project",
    "level": 3,
    "children": [
      "summary:level2:rules.py",
      "summary:level2:service.py"
    ]
  }
]
```

이 작은 예제에는 함수 요약 3개, 파일 요약 2개, 프로젝트 요약 1개가 저장됐다. `policy.py`에는 함수가 없어 독립된 파일 요약은 없으며, 상수 값은 `sellable`의 참조 묶음과 `rules.py` 요약을 통해 전달된다.

이 예제에서 실제 생성한 상위 요약은 다음과 같다.

`service.py` 파일 요약 — 실제 본문

```
The `remaining` function calculates the number of items that can be sold by
subtracting
the number of items held from the number of items on hand and subtracting a
safety
stock. It uses the `sellable` function from the `rules` module, which in
turn uses the
`SAFETY_STOCK` constant from the `policy` module. The `sellable` function
ensures the
result is not negative by using the `max` function. The `label` function
checks if
`on_hand` is less than or equal to 0 and returns 'empty' or 'stocked'
accordingly.
```

프로젝트 요약 — 실제 본문

```
The `service.py` module contains functions to manage inventory and
determine the number
of items that can be sold. The `remaining` function calculates the number
of items
available for sale by subtracting the number of items held from the on-hand
inventory
and applying a safety stock. It uses the `sellable` function from the
`rules.py` module,
which ensures the result is not negative. The `label` function checks if
the inventory
is empty and returns 'empty' or 'stocked'. The `SAFETY_STOCK` constant,
with a value of
3, is defined in the `policy.py` file.
```

앞의 `remaining` 함수 요약에서 빠졌던 값 `3`이 이 프로젝트 요약에는 들어 있다. `rules.py`의 설명까지 함께 다루기 때문이다. 다만 이 한 예제가 상위 요약에서 모든 세부 값이 항상 보존된다는 뜻은 아니다.

RH는 **원문 + 함수 요약 + 파일 요약 + 프로젝트 요약**을 모두 검색 후보로 사용한다. 질문마다 전체 요약 트리를 LLM에 보내지 않는다. 각 단계의 후보를 하나의 목록에서 검색하고, 선택된 일부만 전달한다. 따라서 '계층'은 요약을 만드는 구조를 말하며, 검색할 때 반드시 프로젝트→파일→함수 순으로 내려가는 구현은 아니다.

큰 파일에서 한 파일의 자식 요약들이 모델 입력 한도에 들어가지 않으면, 순서대로 여러 묶음으로 나눠 먼저 요약하고 그 결과를 다시 합친다. 어떤 자식도 임의로 버리지 않으며, 이 중간 요약도 RH·C의 후보에 포함한다. 실제 적용 횟수는 5.5절의 준비 비용 표에 제시한다.

1.8 C — 계층 요약 정리: 같은 자료의 중복과 제목을 줄인다

단계	처리 내용
질문 전 준비	RH와 같은 색인·요약을 공유
사전 LLM 요약	**새 요약 생성은 없음.** RH에서 만든 LLM 요약은 그대로 사용
질문 후 처리	RH와 같은 자료 선택 → 일반 코드로 중복 원문·import 제거와 제목 축약 → LLM 답변. 줄어든 공간은 다시 채우지 않음

C는 **계층 요약 정리 방식**이다. 저장하는 요약과 검색 결과는 RH와 같다. 검색 후 전달 문자열을 만들 때 같은 함수 원문은 한 번만, 같은 파일의 import문도 한 번만 넣고, 긴 제목을 짧게 바꾼다. 예를 들어 원문 제목은 다음처럼 바뀐다.

```
정리 전: ### memory://symbol/service.py::remaining lines 3–4
정리 후: # Code service.py::remaining
```

```
# 정리 후에도 아래 원문 본문은 그대로 유지된다.
from rules import sellable
def remaining(on_hand, held):
    return sellable(on_hand, held)
```

고유 코드·import문·요약 본문은 그대로 보존한다. LLM으로 다시 요약하지 않고, 확보한 여유 공간을 다른 자료로 채우지도 않는다. 위 별도 예제에서 실제 선택 결과는 RH **754토큰**, C **658토큰**이었다. 이는 예제의 입력 구성 수치이며 96문제의 성능 수치는 4.1절에 있다. 본문이 같아도 반복·표시가 바뀌면 LLM의 답은 달라질 수 있다.

1.9 파일 전체 입력과 일곱 방식의 전달 범위

단계	처리 내용
질문 전 준비	검색용 색인·자연어 요약을 만들지 않음. 이 실험에서는 대상 파일 경로를 미리 제공
사전 LLM 요약	**없음**
질문 후 처리	지정된 파일을 처음부터 끝까지 읽음 → 그대로 LLM에 전달해 답변. 다른 파일의 정의는 자동 추가하지 않음

방식	미리 준비하는 자료	이 질문에서 답변에 넣는 자료의 범위
A	함수 이름·파일·줄 위치의 주소록	선택 함수와 같은 파일의 관련 코드; 이 예제에서는 `remaining`
B	A의 주소록 + 함수·상수 원문 + 파일 간 참조	`remaining`, `sellable`, `SAFETY_STOCK`
R0	원문 단일 후보·참조 묶음 + 단어 가중치	검색된 후보를 길이 안에서 조합; 중복·관련성이 낮은 코드가 포함될 수도 있음
RF	R0 원문 후보 + 함수·관련 코드 요약	단어 검색으로 고른 원문·함수 요약 일부
RH	RF 자료 + 파일·프로젝트 요약	여러 단계의 원문·요약 중 선택된 일부

방식	미리 준비하는 자료	이 질문에서 답변에 넣는 자료의 범위
C	RH와 같은 자료	RH와 같은 선택 결과를 중복 제거·출처 축약 후 전달
파일 하나 전체	파일 경로만 제공; 함수 검색용 색인은 사용하지 않음	`service.py`를 처음부터 끝까지 전달: `label`도 포함하지만 다른 파일의 정의는 없음

색인이 차지하는 저장 공간과 LLM에 보내는 토큰은 다르다. A·B는 후보 목록을 보여 주는 '함수 선택' 호출과 원문을 읽는 '답변' 호출이 따로 있다. R0·RF·RH·C와 파일 전체 방식은 질문당 답변 호출만 있다. RF·RH·C의 사전 요약 생성 호출은 이 질의당 횟수와 별도다. 여러 호출의 입력 합계는 총 처리량이고, 컨텍스트 윈도우 점유는 **각 호출의 입력**을 기준으로 봐야 한다.

3.1절의 작은 파일 실험에서는 네 방식의 답변용 코드 상한이 1,024토큰이며, 모델의 전체 컨텍스트는 8,192토큰이다. A·B·R0은 RAPTOR 요약을 사용하지 않는다. 위 예제의 전체 색인·원문 후보·검색 순위는 실제 생성 자료에서 확인할 수 있다.

1.10 RAPTOR의 어떤 아이디어를 적용했나?

RAPTOR 원 논문은 내용을 묶어 요약하고, 요약들을 다시 묶어 상위 요약을 만드는 과정을 반복하며 여러 추상화 단계에서 근거를 찾는 접근을 제안한다. 이 실험은 그중 **요약을 검색 자료로 저장하고, 여러 단계의 요약과 원문을 함께 검색하는 아이디어**를 코드에 적용한 변형이다.

방식	적용한 아이디어	이 실험에서 하지 않는 것
RF	함수·관련 원문을 요약해 원문과 함께 검색	파일·프로젝트까지 재귀적으로 요약하지 않음
RH	함수→파일→프로젝트로 재귀 요약하고 모든 단계에서 검색	임베딩 모델로 비슷한 의미의 내용을 찾아 자동으로 묶는 원 논문의 과정까지 재현하지 않음
C	RH의 요약 구조와 검색을 그대로 사용한 뒤 입력을 정리	새로운 요약 방식이나 RAPTOR 고유 기능이라고 주장하지 않음

여기서는 코드의 참조 관계와 파일 경계로 내용을 묶고, 단어의 출현 빈도와 희소성으로 계산하는 TF-IDF 점수로 검색한다. 따라서 결과는 **이번 구현에 대한 평가**이며, RAPTOR 원 논문 전체의 성능을 검증하거나 반박하는 실험은 아니다. 예제 원문·요약 생성 요청·실제 선택 자료는 예제 기록에 보존했다.

2. UI에서 '현재 프로젝트 색인 생성'을 누르면 누가 무엇을 만드는가?

사용자는 생성 시점과 대상 프로젝트를 정하고, UI는 요청을 전달하며, 백엔드의 색인 생성 작업이 파일 읽기·파싱·자료 구성·저장을 수행한다. RF·RH를 사용할 때는 이 작업이 LLM 서버에 요약을 요청한다. C는 RH의 자료를 공유한다.

여기서 **백엔드**는 화면 뒤에서 실제 파일 처리를 실행하는 프로그램이다. 데스크톱 앱이라면 사용자 컴퓨터의 로컬 프로세스로 둘 수 있다. 현재 확인한 구현은 Python으로 실행하는 **실험용 구축 코드**다. 아래에서 파싱·참조 연결·요약 생성 알고리즘은 실제 코드에 근거하며, **UI 요청 연결·백그라운드 작업·진행 표시·완료된 버전으로 교체하는 절차는 제품에 연결할 설계**다. 이 절을 추가하면서 UI 기능을 구현하거나 새 실험을 실행한 것은 아니다.

2.1 만드는 주체와 역할

주체	맡는 일	직접 만들어 내는 것
사용자	현재 프로젝트에서 생성 버튼을 누름. 사용할 방식은 사용자 선택 또는 앱 설정을 따름	생성 요청
UI	프로젝트 식별 정보와 방식을 백엔드에 전달하고 진행·완료·오류를 표시	요청과 상태 화면
백엔드의 색인 생성 작업	파일 목록을 정하고 읽기, 파싱, 검색 자료 구성, 필요한 LLM 호출, 저장 순서를 관리	프로젝트별 색인 버전과 저장 자료
파서와 분석·검색 코드	`ast.parse`로 문법 구조를 읽고 이름·위치·참조를 추출. 원문 묶음과 TF-IDF 가중치를 계산	주소록, 참조 관계, 원문 후보, 검색용 수치
LLM 서버	백엔드가 보낸 원문 묶음 또는 하위 요약을 읽고 설명 문장을 생성	RF의 함수 요약, RH의 파일·프로젝트 요약 본문

파서와 분석 코드는 백엔드가 호출하는 프로그램 구성요소다. **LLM이 쓰는 것은 요약 본문이며, 파일 경로·함수 주소·줄 번호·요약의 부모/자식 연결은 프로그램이 붙인다.** 이번 실험에서는 Python 클라이언트가 로컬 Qwen 모델 서버에 요청하고 응답 텍스트를 받아 저장했다. 색인을 만드는 단계에서 대상 프로젝트의 함수를 실행해 계산 결과를 구하는 것은 아니다.

2.2 버튼 클릭부터 '사용 가능'까지의 처리 순서

다음은 현재 구축 로직을 하나의 버튼 작업으로 연결하는 흐름이다. **기본 입력은 디스크에 저장된 현재 프로젝트 파일**로 잡는다. 편집기의 미저장 내용까지 포함하려면 그 내용을 백엔드에 전달하는 연동을 별도로 넣어야 한다.

순서	실행 주체	처리하는 내용과 다음 단계로 넘기는 결과
1. 요청 접수	UI → 백엔드	프로젝트 경로·방식·요약 모델 설정을 전달한다. 백엔드는 작업 ID를 만들고 UI에는 작업 시작 상태를 돌려준다.
2. 대상 파일 확정	백엔드	프로젝트 파일을 수집하고, 이번 생성에 사용할 파일 내용과 버전을 고정한다. 현재 파서는 Python 파일을 대상으로 한다. 제외 폴더와 지원 언어는 앱의 수집 규칙으로 정한다.
3. 코드 구조 읽기	파서 + 분석 코드	파일을 AST로 읽는다. A에는 함수·클래스 등의 이름·경로·줄·URI를, B 이후의 참조 자료에는 함수·상수 원문·import·참조 이름을 만든다.
4. 연결과 원문 후보 구성	분석 코드	`from … import …` 등으로 프로젝트 안의 참조 대상을 연결한다. R0·RF·RH·C에는 함수와 관련 코드를 묶은 후보와 단일 원문 후보를 만든다.
5. 필요한 요약 생성	백엔드 → LLM 서버 → 백엔드	RF·RH·C일 때 함수별 원문 묶음을 요약한다. RH·C는 그 요약들을 파일별로 재요약하고, 파일 요약들을 프로젝트 요약으로 만든다. A·B·R0에는 이 단계가 없다.
6. 저장과 검색 준비	백엔드 + 검색 코드	원문·주소·참조·요약을 저장하고 연결 정보를 점검한다. R0·RF·RH·C는 후보를 비교할 단어 가중치와 벡터를 준비한다. 디스크 저장과 메모리 준비의 현재 구현 차이는 2.4절에 설명한다.
7. 완료 버전 반영	백엔드 → UI	검증된 결과를 현재 프로젝트의 사용 가능한 색인으로 지정한다. UI는 처리한 파일·함수·요약 수, 완료 시각과 실패 항목을 보여 준다.

UI는 장시간 작업의 결과를 기다리며 화면을 멈추게 하기보다, 백엔드 작업의 상태를 받아 표시하도록 연결한다. 파일 파싱·함수 요약·파일 요약처럼 **현재 단계와 처리 개수**를 보여 주면 무엇을 하고 있는지 알 수 있다. 실험의 요약 시간은 5.5절에 있지만, 그것을 사용자의 현재 프로젝트에 대한 예상 완료 시간으로 그대로 사용할 수는 없다.

2.3 선택한 방식에 따라 어디까지 만들어야 하나?

방식	버튼 작업에서 준비할 자료	준비 단계의 요약 LLM 호출
A — 현재 색인	함수 등의 이름·파일·줄 위치·URI 주소록	없음
B — 파일 간 확장	A 주소록 + 함수·상수 원문 + import와 파일 간 참조 관계	없음
R0 — 원문 검색	참조 관계 + 단일 원문·관련 코드 묶음 후보 + 검색용 가중치	없음
RF — 함수 요약	R0 자료 + 함수별 원문 묶음의 요약과 출처 연결 + 요약 후보의 검색 준비	함수 요약 생성 시 호출
RH — 계층 요약	RF 자료 + 파일·프로젝트 요약과 부모/자식 연결 + 상위 요약의 검색 준비	함수 요약 후 파일·프로젝트 요약 생성 시 호출
C — 계층 요약 정리	RH와 같은 자료를 공유	RH 구축 비용을 공유하며 C를 위해 새로 요약하지 않음
파일 전체	검색 전용 색인 불필요	없음

예를 들어 A만 사용한다면 전체 요약 트리를 만들 필요가 없다. RF와 RH를 함께 비교한다면 함수 요약은 한 번 만들고 공유할 수 있다. 실험의 `assets.json`에는 여러 방식을 비교할 자료가 함께 들어 있지만, UI에서 한 방식만 사용할 때 모든 방식의 자료를 반드시 만들 필요는 없다.

질문이 들어온 뒤에만 하는 작업도 있다. A·B의 '질문에 맞는 출발 함수를 LLM으로 선택', B의 '선택 함수에서 참조 확장', R0·RF·RH·C의 '질문과 후보의 점수 비교·토큰 한도 안 선택'은 버튼을 누를 때 완료할 수 없다. 특히 **C의 중복 제거는 RH가 그 질문에서 고른 자료에 적용하는 처리**다. 별도의 'C 압축 색인'을 생성하는 단계가 아니다.

2.4 실제 코드는 무엇을 저장하고, 무엇을 메모리에 준비하나?

자료	실제 구현에서 만드는 곳	현재 저장 또는 사용 형태
A의 주소록	`build_index(root)`가 파일을 파싱하여 객체 반환	반환만으로 파일이 생기지는 않는다. `write_index(root, output)`가 JSON으로 저장하며 기본 파일명은 `symbol_index.json`이다.
원문·참조·원문 후보	`source_nodes(root)` → `raw_items(nodes)`	큰 파일 실험은 `index`, `nodes`, `raw`를 묶어 `assets.json`에 저장한다.
함수·파일·프로젝트 요약	큰 파일 요약 빌더가 원문 후보 또는 하위 요약을 LLM에 전달	생성 중에는 `summary_nodes.jsonl`에 기록하고, 프로젝트별 `assets/<project>.json`과 전체 `assets.json`에 요약을 합친다. 호출·출처·자식 ID도 보존한다.
생성 완료 내역	요약 빌더	`build_manifest.json`에 생성 수·토큰·시간·자료 해시를 기록한다. 이는 현재 실험의 기록 형식이다.

자료	실제 구현에서 만드는 곳	현재 저장 또는 사용 형태
TF-IDF 가중치와 후보 벡터	`Retriever` 검색 코드	현재는 평가 실행 시작 시 메모리에 계산하고 질문들에 재사용한다. 위 JSON에 가중치·벡터를 영구 저장하는 구현은 아니다.

큰 파일 RF·RH·C 실험은 앞선 원문 구축 결과를 `base_assets.json`으로 가져와 요약을 추가했다. **UI에서는 '원문 구축 → 필요한 요약 → 검색 준비'를 하나의 작업으로 연결하면 된다.** 앱을 다시 켤 때 저장 자료를 읽고 메모리의 검색 준비를 다시 수행하거나, 별도의 검색 저장 형식을 구현할 수 있다. 어떤 경우든 '저장 완료'와 '바로 검색 가능한 상태'를 구분해야 한다.

1.2절의 세 파일 예제로 보면, 백엔드가 `service.py`에서 `remaining`의 위치를 얻고, 참조 분석으로 `rules.py`의 `sellable`, `policy.py`의 `SAFETY_STOCK = 3`까지 연결한다. RF를 쓰면 **이 연결된 원문 묶음과 요약 지시문을 백엔드가 LLM에 보내고**, 응답 문장을 출처 목록과 함께 저장한다. RH를 쓰면 **백엔드가 파일별 묶음을 정하고 LLM이 그 묶음의 설명을 작성**한다. 사용자는 이 과정의 시작과 완료를 UI에서 확인한다.

2.5 코드가 바뀌거나 생성이 실패했을 때

사용자가 원하는 시점에 다시 생성하려면 **어떤 프로젝트의 어느 파일 내용으로 만든 색인인지**를 함께 관리해야 한다. A는 줄 위치가 바뀌면 다른 원문을 읽을 수 있고, B·R0는 저장한 원문이 오래될 수 있으며, RF·RH·C는 그 원문에서 만든 요약까지 오래될 수 있다.

제품에 연결할 때의 첫 구현은 버튼을 누를 때 선택 방식의 자료를 새 버전으로 만들고, **완성된 결과만 현재 색인으로 교체**하는 흐름으로 정할 수 있다. 생성 중 파일이 바뀌면 같은 버전의 파일 복사본을 끝까지 쓰거나 작업을 다시 시작하는 규칙이 필요하다. 변경된 부분만 갱신하는 기능을 추가한다면 바뀐 함수뿐 아니라 그 함수를 포함한 원문 묶음과 상위 요약도 갱신 대상이다.

파싱 오류·파일 읽기 실패·LLM 호출 실패는 작업 상태에 남기고, 실패한 결과로 마지막 정상 색인을 조용히 교체하지 않도록 한다. 일부만 사용할 경우에는 '부분 완료'와 빠진 항목을 표시하는 정책을 정해야 한다. 현재 실험에는 파일 해시와 요약 요청 해시를 확인하는 로직이 있지만, **UI의 자동 변경 감지·증분 갱신·취소·완료 버전 전환이 구현됐다는 뜻은 아니다.** 실험의 요약 출력 한도 처리 규칙은 3.3절과 5.5절에 별도로 보존했다.

현재 파서는 Python 중심이고, 파일 간 연결도 코드에서 읽어 낸 제한적인 정적 참조 분석이다. 다른 언어나 복잡한 import·동적 호출을 지원하려면 해당 분석기를 추가해야 한다. 또한 A 주소록과 참조 분석의 현재 파일 수집 규칙이 완전히 같지는 않으므로, UI 작업에서는 동일한 대상 파일 목록을 쓰도록 맞춰야 한다.

실제 구현 근거: 주소록 생성과 저장 · 참조 분석·원문 후보·검색 코드 · 원문 자료 저장 · 요약 생성과 저장 · C의 질문별 정리.

3. 실험 조건과 지표를 읽는 법

작은 파일의 네 방식 비교, 같은 코드를 확장한 큰 파일의 네 방식 비교, 그 큰 파일에 RF·RH·C를 추가한 비교를 구분한다. **새로 생성한 답변과 이전 실행에서 보존한 답변**을 아래에 명시했다.

3.1 작은 파일: A·B·R0·파일 전체

기존 A·B·R0의 **864개 실제 답변**을 보존하고, 파일 전체 방식에서 **96문제 × 3회 = 288개 답변을 새로 생성**했다. 합계는 **1,152개 실제 평가 답변**이다. 토큰만 계산했던 이전 비교는 이 실제 실행 결과로 대체했다.

항목	실제 조건
질문·원본 코드	기존과 같은 6개 합성 Python 프로젝트, 각 5파일·16문제

항목	실제 조건
파일 전체 입력	질문 대상 함수가 정의된 파일을 처음부터 끝까지 그대로 전달. 81문제는 `service.py`, 15문제는 `rules.py`
파일 선택 조건	파일 경로를 미리 제공. 파일을 찾는 과정은 평가하지 않음
모델·장비	Qwen2.5-Coder-7B-Instruct Q5_K_M · Apple M3 Pro · 메모리 36 GiB
공통 설정	같은 질문·답변 지시문, temperature 0, seed 1, 답변 최대 128토큰
길이 제한	답변용 코드 최대 1,024토큰, 전체 문맥 8,192토큰. 파일 전체도 모두 상한 안에 들어가 자르지 않음
실행·캐시	방식별 각 질문 3회. 같은 질문·반복 순서, 모델 입력 계산 재사용 끔
정답 확인	코드 실행·호출 관계로 96개 기준 답 재확인. JSON 값·자료형·요구한 목록 순서까지 일치해야 정답

개발용 8문제는 실행 확인에만 사용하고 평가 점수에서 제외했다. 각 문제·방식의 최종 답은 세 반복 모두 같았다. 정답 수는 서로 다른 96문제를 기준으로 표시한다.

비교의 범위: 파일 전체 방식은 관련 파일을 이미 알고 시작하는 조건이다. 반면 A·B는 후보에서 출발 함수를 고르는 비용과 오류도 포함한다. 또한 B·R0은 여러 파일의 원문을 모으므로 제공하는 정보 범위가 다르다. 이 실험은 그런 차이를 포함한 구현 전체의 비교다.

시간의 한계: A·B는 같은 세션에서 교차 실행했고, R0과 이번 파일 전체 방식은 각각 별도 세션에서 측정했다. 모델·실행 파일·설정은 같지만 장비 부하·열 상태까지 통제하지는 못했다. 시간 차이는 관측값이며 방식만의 순수한 속도 효과로 확정하지 않는다.

3.2 큰 파일: 원래 기능을 유지하고 보조 함수를 추가

기존 **같은 96문제 × 3회 × 4방식 = 1,152개 답변을 새로 생성**했다. 개발용 32개 답변과 예열은 점수에서 제외했다. 이전의 작은 파일 결과는 보존하고 비교에 함께 제시한다.

항목	이번 조건
파일 크기	평가용 6프로젝트 × 5파일. 각 파일 4,099~4,157토큰, 533~553줄
커진 부분	파일마다 관계없는 보조 함수 57~60개 추가. 12종류의 실행 가능한 함수 패턴을 이름을 달리해 반복한 합성 자료
유지한 부분	기존 함수·상수의 원문, import, 참조 관계, 질문·정답. 96개 정답을 다시 실행하여 확인
원래 코드 위치	파일 앞·중간·뒤에 배치. 평가 프로젝트를 각 위치에 2개씩 배정
A·B·R0	기존 선택·확장·검색 구현 그대로 사용. 답변용 코드 자료 최대 1,024토큰
파일 하나 전체	대상 함수가 정의된 파일 경로를 미리 제공하고, 그 파일 전체를 원문 그대로 전달. 다른 파일은 자동으로 추가하지 않음
공통 모델·문맥	기존과 같은 Qwen2.5-Coder-7B-Instruct Q5_K_M, M3 Pro, 컨텍스트 8,192토큰
생성 조건	같은 질문·답변 지시문, temperature 0, seed 1, 답변 최대 128토큰, 입력 계산 재사용 끔

항목	이번 조건
실행 순서	같은 세션에서 질문마다 네 방식을 번갈아 실행. 프로젝트·반복마다 각 방식이 1~4번째에 같은 횟수로 등장

길이 제한의 차이: 기존 작은 파일은 원문 전체도 1,024토큰 안에 들어갔다. 이번 파일 전체 방식에는 그 코드 한도를 적용하지 않아, 4천여 토큰의 원문을 자르지 않고 모두 전달했다. 네 방식의 전체 컨텍스트 한도는 동일한 8,192토큰이다. 따라서 이번 비교는 **같은 양의 코드를 준 비교가 아니라, '일부를 고르는 전략'과 '파일 전체를 주는 전략'의 비교**다.

같은 질문과 원래 동작을 유지했지만, **파일 크기만 변한 것은 아니다.** 관계없는 함수와 검색 후보가 늘었고, 원래 코드의 줄 위치도 달라졌다. 그래서 이 결과는 '짧은 관련 코드를 포함한 파일에 다른 기능이 많이 추가된 상황'에 대한 실험이다. 관련 함수 자체가 수천 줄로 길어지는 경우는 평가하지 않았다.

큰 파일 네 방식은 같은 세션에서 번갈아 측정했다. 작은 파일은 이전 세션의 기록이므로, 작은 파일과 큰 파일 사이의 시간 변화에는 장비 상태 차이가 섞일 수 있다. 토큰은 실제 입력 문자열과 같은 모델의 토크나이저로 확인한 값이다.

3.3 같은 큰 파일에 RF·RH·C 추가

항목	고정한 조건
원문·질문	3.2절과 바이트까지 동일한 큰 파일, 동일한 96문제·정답. 평가용 6개 합성 프로젝트, 파일당 4,099~4,157토큰
새 평가	RF·RH·C 각각 96문제 × 3회 = **864개 실제 답변**
비교 기준	3.2절의 A·B·R0·파일 전체 **1,152개 실제 답변을 보존**하여 함께 제시
모델·생성	같은 Qwen2.5-Coder-7B Q5_K_M, M3 Pro, 컨텍스트 8,192, 출력 128, temperature 0, seed 1, 입력 계산 재사용 끔
정답 판정	JSON 정답 값·자료형·요구한 목록 순서까지 일치해야 정답. 형식 실패와 출력 한도 종료도 오답에 포함
근거 선택	RF·RH는 최대 1,024토큰. C는 RH의 동일 선택 결과를 정리하고 여유 공간을 채우지 않음. 파일 전체는 원문 전체를 전달
요약	질문·정답을 입력하지 않고 원문에서 새로 생성. 함수 요약은 출력 최대 256토큰·100단어 이내 지시, 상위 요약도 같은 지시·한도
실행 순서	매 반복에서 세 방식의 6가지 순서를 각각 16번씩 배치. 질문 블록 순서는 고정 시드로 무작위화
별도 개발 확인	평가와 다른 8문제 × 3방식 = 24답변. 주 점수에서 제외

RF·RH·C는 같은 세션에서 번갈아 실행했지만, A·B·R0·파일 전체는 앞선 세션이다. **새 세 방식 사이의 시간 비교는 같은 세션이며, 이전 네 방식과의 시간 차이는 세션 상태까지 통제한 결과가 아니다.** RF·RH·C의 검색 규칙과 답변 지시문은 01번 실험의 해당 방식을 유지했다. 토큰 계산과 모델 파일은 3.2절의 큰 파일 실험과 같다.

파일 단위 상위 요약이 입력 한도에 들어가지 않을 때만 중간 묶음 요약을 추가하도록 **생성 전에 규칙을 고정**했다. 자식 요약을 잘라 버리지 않는다. 이 처리까지 포함한 변형이며, 원 논문 구현 전체를 재현한 것은 아니다.

이번 평가용 데이터에서는 모든 파일의 자식 요약이 입력 한도 안에 들어가 중간 묶음 요약은 0개였고, 함수→파일→프로젝트의 3단계를 사용했다.

이전 요약 준비 코드는 요약 실패 시 중단한다. 이번에는 사전에 정한 규칙에 따라 **출력 한도에서 끝난 비어 있지 않은 요약도 생성 기록에 상태를 남기고 후보로 보존**한다. 생성 조건의 이 차이도 공개하며, 해당 횟수·비용과 실제 답변에 선택됐는지는 5.5절에 제시한다.

3.4 정답률·시간·근거·토큰의 뜻

대표 응답 시간은 문제마다 3회 중앙값을 구한 뒤 96문제에 걸쳐 평균했다. 검색 또는 제공된 파일 읽기 시작부터 최종 답변까지 측정하며, 토큰 계산과 작은 호출 로그 기록도 포함한다. 모델 시작·사전 색인 준비·채점은 제외한다. **p95**는 각 방식의 288요청 중 약 95%가 끝나는 시간이다.

근거 포함률은 문제별로 미리 지정한 코드 목록 중 실제 전달한 원문의 비율이다. 목록을 다 포함해도 답을 맞힌다는 뜻은 아니다. 파일 전체는 한 파일의 모든 코드를 포함하지만 다른 파일의 정의는 포함하지 않는다.

'코드 자료' 토큰은 답변에 첨부한 문자열 전체를 센 값이다. A·B·R0은 원문 조각뿐 아니라 함께 붙인 제목·import문·코드 표시까지 포함하며, 파일 전체 방식은 파일 원문 그대로다. '전체 입력'에는 여기에 질문·지시문·대화 형식과, A·B의 함수 선택 호출 입력까지 포함한다.

RF·RH·C도 각 문제의 세 출력이 모두 같았다. 따라서 정답 수는 반복을 제외한 96문제 기준이다. 대표 시간은 문제별 세 번의 중앙값을 구한 뒤 평균한 값이며, p95는 전체 요청의 95%가 끝나는 시간이다. '근거'는 코드·요약·출처 제목 등을 포함한 선택 자료이고, '총 입력'에는 질문·지시문·대화 형식과 A·B의 함수 선택 호출도 포함한다. 사전 요약 생성과 출력 토큰은 질문당 입력 표에서 제외하고 5.5절에 별도로 제시한다.

본문의 **%p**는 두 정답률을 뺀 차이다. 예를 들어 26.0%와 15.6%의 차이는 10.4%p다. 반복 답변을 별개의 문제로 세지 않으며, 통계 계산과 비교별 불확실성은 부록 A.4~A.6에 모았다.

4. 정답률·시간·파일 크기에 따른 결과

4.1 큰 파일: 일곱 방식의 핵심 결과

방식	정답 / 96	정답률	대표 응답 시간	p95
현재 색인(A)	17	17.7%	2.62초	4.89초
파일 간 확장(B)	20	20.8%	3.50초	5.41초
원문 검색(R0)	25	26.0%	5.00초	5.90초
함수 요약(RF)	15	15.6%	5.80초	7.27초
계층 요약(RH)	16	16.7%	5.79초	7.16초
계층 요약 정리(C)	16	16.7%	5.23초	7.10초
파일 전체	15	15.6%	14.35초	18.69초

이 96문제의 관측값에서 **정답 수가 가장 많은 방식은 원문 검색(R0)(25/96)**이고, **질문당 총 입력이 가장 적은 방식은 현재 색인(A)(500.9토큰)**이다. 정확도와 입력량 중 무엇을 우선하느냐에 따라 선택이 달라진다. 통계적 불확실성은 부록 A.6, 사전 요약 비용은 5.5절에서 함께 살펴본다.

출력 최대 128토큰에서 끝난 평가 호출: **A 0회 · B 0회 · R0 0회 · RF 3회 · RH 3회 · C 0회 · 파일 전체 21회**. 이 응답들은 오답으로 포함했으며 성공한 재시도로 대체하지 않았다. 입력 잘림이나 전송 실패 여부는 검증 기록에서 확인했다. 파일 전체 방식의 21회 제한 종료라는 기존 실험 조건도 그대로 유지된다.

- **함수 요약 추가(R0→RF):** 정답 25→15개(-10개). 보정 검정에서 정확도 차이를 확정하지 못했다.
- **파일·프로젝트 요약 추가(RF→RH):** 정답 15→16개(+1개). 보정 검정에서 정확도 차이를 확정하지 못했다.
- **입력 정리(RH→C):** 정답 16→16개(0개). 보정 검정에서 정확도 차이를 확정하지 못했다.

네 방식만의 기존 큰 파일 상세표도 부록 A.1에 유지했다. 실제 답변 사례는 부록 B에서 확인할 수 있다.

4.2 작은 파일: 네 방식의 실제 결과

지표	현재 색인(A)	파일 간 확장(B)	원문 검색(R0)	파일 하나 전체
정답 수 / 96문제	17	22	26	17
정답률	17.7%	22.9%	27.1%	17.7%
대표 응답 시간	2.35초	3.17초	4.38초	1.81초
느린 요청의 시간: p95	3.48초	4.81초	5.25초	2.51초
LLM에 전달한 코드 자료 평균	97.1토큰	348.5토큰	1016.2토큰	168.6토큰
질문당 전체 입력 평균	409.1토큰	660.2토큰	1164.7토큰	316.7토큰
지정 원문 근거의 평균 포함률	28.8%	72.4%	100.0%	30.6%
지정 원문 근거를 모두 포함한 비율	14.6%	49.0%	100.0%	14.6%
실제 평가 답변 수	288회	288회	288회	288회
질문당 LLM 호출 수	2회	2회	1회	1회

A와 파일 전체는 정답 수가 같아도 맞힌 문제가 모두 같지는 않았다. A만 맞힌 문제가 4개, 파일 전체만 맞힌 문제가 4개였다. 따라서 정답 수가 같다는 결과만으로 두 방식의 품질이 동등하다고 확정할 수는 없다.

4.3 작은 파일에서 큰 파일로 바뀌면

방식	정답 수: 작은 파일 → 큰 파일	코드 토큰: 작은 파일 → 큰 파일	총 입력: 작은 파일 → 큰 파일
현재 색인(A)	17 → 17 / 96	97.1 → 97.2	409.1 → 500.9
파일 간 확장(B)	22 → 20 / 96	348.5 → 366.0	660.2 → 769.4
원문 검색(R0)	26 → 25 / 96	1016.2 → 1016.1	1164.7 → 1164.7
파일 하나 전체	17 → 15 / 96	168.6 → 4117.6	316.7 → 4265.8

파일 전체의 코드 입력은 작은 파일 때보다 **24.4배**가 되었지만, B는 **1.05배**, R0은 **1.00배**였다. 이번 조건에서 토큰 비교가 역전된 직접적인 이유다. B는 관련 코드만 확장하고, R0은 기존 1,024토큰 한도 안에서 검색 결과를 담기 때문이다.

작은 파일 크기가 중요한 이유였지만, B·R0이 인덱스만 전달한다는 이해는 수정해야 한다. 색인은 코드를 찾는 자료다. B는 그 색인을 따라 여러 파일의 함수·상수 원문을 모아 보내고, R0은 검색된 원문 조각과 참조 묶음을 보낸다. B에는 출발 함수를 고르는 별도 LLM 호출도 있다. R0은 일부가 겹치는 원문 묶음을 함께 넣을 수 있으며, 점수 하한 없이 1,024토큰까지 후보를 담는다.

작은 파일 실험의 파일 전체는 평균 **168.6토큰**이었다. 그보다 넓은 범위를 읽은 B는 **348.5토큰**, R0은 **1016.2토큰**의 코드 자료를 보냈다. 비교 대상 파일이 작고, 가져오는 원문 범위·표시·중복·호출 수가 달랐기 때문에 토큰이 더 들었던 것이다.

4.4 01번 보고서와 비교할 때: 같은 48문제로 맞춰야 한다

01번 보고서의 최초 실험은 **48문제**, 최근 비교는 그 문제와 추가 48문제를 합친 **96문제**다. 정답 개수를 분모 없이 비교하면 개선 폭을 오해할 수 있다. 아래는 질문 문장과 정답이 같은 최초 48문제로 맞춘 비교다.

방식	최초 작은 파일: 같은 48문제	이번 큰 파일: 같은 48문제	나머지 48문제: 큰 파일
현재 색인(A)	8	8	9
파일 간 확장(B)	12	10	10
원문 검색(R0)	12	10	15
함수 요약(RF)	9	6	9
계층 요약(RH)	9	7	9
계층 요약 정리(C)	10 (후속 1회 재확인)	7	9
파일 전체	최초 실험 없음	6	9

최초 실험에는 C와 파일 전체 방식이 없었다. C의 48문제 수치는 01번 보고서에 기록된 **후속 재확인 1회**이며, 이번에는 3회 실행했다. 파일을 키울 때 원래 기능은 유지했지만 관계없는 함수·검색 후보와 줄 위치도 함께 바뀌었다. 요약 역시 커진 파일에서 새로 생성했다. 따라서 아래 차이는 파일 크기 하나만의 인과 효과가 아니다. 최초 실험의 시간은 일부 호출 비용을 공유·합산했으므로 이번 실측 시간과 직접 비교하지 않는다.

01번 보고서 2차의 **A 9개·RH 10개·C 11개**는 위 표의 '나머지 48문제'에 해당한다. 이 문제들끼리만 맞춰 비교하면 다음과 같다. RF는 당시 이 별도 48문제로 평가하지 않았다.

01번 보고서의 2차와 동일한 별도 48문제	당시 작은 파일 정답	이번 큰 파일 정답
현재 색인(A)	9	9
계층 요약(RH)	10	9
계층 요약 정리(C)	11	9

5. 토큰·컨텍스트·준비 비용

컨텍스트 절약은 한 번의 요청에서 차지하는 입력을, **처리량 절약**은 질문을 마칠 때까지 여러 호출에 쓴 입력의 합을 본다. 여기에 질문 전에 요약을 만드는 비용을 따로 더해야 한다.

5.1 큰 파일: 질문당 입력과 한 호출의 점유량

예를 들어 A는 함수 선택과 답변을 **별도 호출**로 처리한다. 두 호출에 쓴 입력을 더한 것이 질문당 총 입력이며, 그 평균은 500.9토큰이다. 질문별로 더 큰 호출을 고른 뒤 평균한 값은 342.0토큰이다. 342.0은 두 호출의 입력을 단순히 평균한 값이나, 모든 요청 중 최대값을 뜻하지 않는다. 여기서 비용은 토큰과 처리 시간으로 비교하며, 금전 요금은 측정하지 않았다.

방식	평균 근거 토큰	평균 총 입력 토큰	파일 전체 대비 총 입력 절감
현재 색인(A)	97.2	500.9	88.3%
파일 간 확장(B)	366.0	769.4	82.0%

방식	평균 근거 토큰	평균 총 입력 토큰	파일 전체 대비 총 입력 절감
원문 검색(R0)	1016.1	1164.7	72.7%
함수 요약(RF)	1015.6	1163.9	72.7%
계층 요약(RH)	1016.2	1164.5	72.7%
계층 요약 정리(C)	804.6	952.9	77.7%
파일 전체	4117.6	4265.8	0.0%

전체 파일 대비 입력 절감률은 **1 – 방식의 평균 총 입력 / 전체 파일의 평균 총 입력**으로 계산했다. RF·RH도 1,024토큰 상한을 두므로 큰 파일 전체보다 짧다는 사실 자체는 예상되는 결과다. 요약의 가치 여부는 같은 상한의 R0보다 얼마나 더 정확한지와 준비 비용까지 함께 봐야 한다.

방식	평균 선택 호출 입력	평균 답변 호출 입력	질문 내 최대 호출 입력의 평균	8,192토큰 중 평균 점유
현재 색인(A)	254.6	246.3	342.0	4.17%
파일 간 확장(B)	254.6	514.8	591.1	7.22%
원문 검색(R0)	0.0	1164.7	1164.7	14.22%
함수 요약(RF)	0.0	1163.9	1163.9	14.21%
계층 요약(RH)	0.0	1164.5	1164.5	14.21%
계층 요약 정리(C)	0.0	952.9	952.9	11.63%
파일 전체	0.0	4265.8	4265.8	52.07%

RF·RH·C는 질문당 답변 호출 1회이므로 답변 입력=총 입력=한 호출 입력이다. A·B의 선택 호출과 답변 호출은 별도 컨텍스트를 사용하므로 누적 입력을 한 번의 점유량으로 읽으면 안 된다. '최대 호출 입력의 평균'은 각 질문에서 가장 큰 호출을 고른 뒤 평균한 값이다. 위 점유율은 입력만이며 출력 공간은 별도 확보한다.

함수 선택 호출과 답변 호출은 별도 문맥을 사용한다. 따라서 누적 입력량과 한 번에 차지하는 컨텍스트를 구분해야 한다. 답변용 출력 128토큰, 함수 선택용 출력 96토큰은 각 호출에서 별도로 확보했다.

5.2 작은 파일: A는 한 호출을 줄여도 누적 입력은 늘 수 있다

아래 토큰은 모두 **실제 LLM 요청에 들어간 입력**이다. 함수 선택 입력과 최종 답변 입력에는 각각 질문·지시문·대화 형식이 포함되어 있다. 출력 토큰은 포함하지 않는다. 비율 열을 제외한 단위는 토큰이다.

방식	함수 선택 입력	최종 답변 입력	누적 입력	가장 큰 한 호출 입력	8,192토큰 중 입력 점유
현재 색인(A)	163.0	246.2	409.1	252.6	3.08%
파일 간 확장(B)	163.0	497.3	660.2	498.5	6.09%
원문 검색(R0)	0.0	1164.7	1164.7	1164.7	14.22%
파일 하나 전체	0.0	316.7	316.7	316.7	3.87%

'가장 큰 한 호출 입력'은 질문별로 두 호출 중 큰 값을 고른 뒤 평균한 것이다. R0과 파일 전체는 한 호출만 있다. A의 이 값은 파일 전체보다 **20.2% 작았다.** 따라서 이번 자료에서는 **한 번에 차지하는 컨텍스트를 줄이는 목적과, 여러 호출의 총 입력량을 줄이는 목적의 결과가 달랐다.** 출력 공간은 답변 최대 128토큰, 함수 선택 최대 96토큰을 별도로 확보했다.

모든 평균은 같은 96문제를 같은 비중으로 계산했다. 토큰 증감률은 각 방식의 입력 합계를 파일 전체 방식의 입력 합계와 비교했다. B·R0은 여러 파일의 원문과 식별용 제목을 포함하므로 파일 하나보다 토큰 수가 커질 수 있다.

5.3 파일 전체 대비 증감: 작은 파일과 큰 파일

작은 파일 기준

아래는 **각 색인 방식 – 파일 전체 방식**의 차이다. 시간은 양수일수록 더 오래 걸렸고, 토큰 증감률은 음수일수록 덜 보냈다는 뜻이다.

파일 전체와 비교한 방식	정답률 차이	관측 시간 차이	코드 토큰 증감	총 입력 토큰 증감
현재 색인(A)	+0.0%p	+0.54초	-42.4%	+29.2%
파일 간 확장(B)	+5.2%p	+1.37초	+106.7%	+108.5%
원문 검색(R0)	+9.4%p	+2.57초	+502.8%	+267.8%

A가 전달한 코드 자료는 평균 **97.1토큰**, 파일 전체는 **168.6토큰**이었다. A는 코드 자료를 **42.4% 줄였지만**, 함수 선택 호출을 포함한 총 입력은 **29.2% 많았다.** 이번 기준 파일은 104~199토큰으로 작은 편이므로, 이 비율을 큰 실제 파일에 그대로 적용하지 않는다.

큰 파일 기준

아래는 **각 방식 – 파일 전체**다. 음수는 토큰이 적거나 시간이 짧다는 뜻이다. 정답률의 %p는 두 비율의 차이다.

파일 전체 대비	코드 토큰 증감	총 입력 증감	한 호출 입력 증감	정답률 차이	시간 차이
현재 색인(A)	-97.6%	-88.3%	-92.0%	+2.1%p	-11.73초
파일 간 확장(B)	-91.1%	-82.0%	-86.1%	+5.2%p	-10.85초
원문 검색(R0)	-75.3%	-72.7%	-72.7%	+10.4%p	-9.35초

큰 파일에서는 A·B·R0 모두 파일 전체보다 총 입력을 줄였다. 다만 A·B·R0의 코드 자료는 1,024토큰 이하로 제한하고 파일 전체는 4천여 토큰을 보내므로, **코드 토큰 절감 자체는 설정상 예상되는 결과**다. 실제 실험에서 확인할 핵심은 그 절감과 함께 정답률·검색 비용·응답 시간이 어떻게 달라지는가다.

5.4 시간은 어느 단계에 들었나?

작은 파일 — 요청당 평균

단계: 요청당 평균	A	B	R0	파일 전체
출발 함수 선택	0.920초	0.919초	없음	없음
원문 준비·길이 확인	0.005초	0.016초	0.085초	0.003초
LLM의 최종 답변 처리	1.425초	2.239초	4.326초	1.808초

단계별 시간은 288요청 전체 평균이므로 3.4절의 '문제별 중앙값의 평균'과 합계가 정확히 같을 필요는 없다. 파일 전체 방식은 제공된 파일을 읽고 한 번 답한다. A·B는 출발 함수 선택 호출이 추가되며, R0은 그 호출이 없지만 더 많은 원문을 답변 모델에 보낸다. 별도 세션이라는 시간 비교의 한계는 여기에도 적용된다.

큰 파일 — 요청당 평균

단계: 요청당 평균	A	B	R0	파일 전체
출발 함수 선택	1.189초	1.189초	0.000초	0.000초
원문 검색·읽기·구성·길이 확인	0.009초	0.012초	0.739초	0.016초
최종 답변 처리	1.424초	2.302초	4.274초	14.356초

단계별 표는 요청 전체 평균이므로, 문제별 중앙값을 사용한 대표 시간과 합계가 정확히 같을 필요는 없다. 사전 색인·참조 그래프·검색 벡터 생성 시간은 질의 시간에서 제외했다. 파일이 커지면서 늘어난 후보를 확인하고 토큰 길이를 계산하는 시간은 질의 시간에 포함했다.

5.5 사전 요약 생성 비용과 재사용

준비 단계: 평가용 6개 프로젝트	실제 생성 수	입력 합계	출력 합계	한 호출 최대 입력	모델 호출 시간 합계
함수 요약	1848	372,466	174,831	835	140.6분
파일 안 중간 묶음 요약	0	0	0	0	0.0분
최종 파일 요약	30	213,514	5,428	7,433	15.5분
프로젝트 요약	6	6,222	684	1,272	0.8분

평가용 요약 생성은 총 **1,884회**, 입력 **592,202토큰**, 출력 **180,943토큰**, 모델 호출 시간 합계 **156.8분**이었다. 단일 서버에서 순서대로 생성했으며, 이 시간은 각 요청의 토큰 계산부터 모델 응답을 받을 때까지의 합계다. 서버 시작·파일 준비·그룹 구성·로그 쓰기까지 포함한 전체 준비 시간은 더 길다. 개발용 프로젝트는 별도 314회이고 위 표에서 제외했다.

사전 요약 중 한 호출의 최대 입력은 **7,433토큰**, 8,192토큰의 **90.7%**였다. 출력 공간은 별도로 필요하다. 질문할 때의 입력이 작아졌다고 해서 색인을 만들 때도 같은 작은 컨텍스트만 필요하다는 뜻은 아니다.

RF에는 함수 요약까지만 필요하다. RH·C는 그 요약을 공유하고 파일·프로젝트 요약을 추가로 사용한다. 세 방식 각각의 비용인 것처럼 공유 요약 비용을 세 번 더하지 않았다. 사전 요약에서 출력 한도에 도달한 호출은 **13회**이며, 중단 시점까지 생성된 요약 본문을 그대로 저장하고 상태를 표시하도록 사전에 정했다.

출력이 중단된 요약 또는 그것을 재료로 삼은 상위 요약이 실제 선택된 평가 호출은 **RF 0/288회 · RH 0/288회 · C 0/288회**였다. 이는 선택된 자료의 출처 연결까지 따라가 확인한 횟수다. 요약의 생성 상태와 실제 답변에 전달됐는지를 구분해 기록했다.

A·B·R0·파일 전체에는 이 사전 LLM 요약 생성 단계가 없다. 색인·참조 관계·단어 가중치를 계산하는 컴퓨터 처리 비용과, 질문 중 사용하는 LLM 호출은 별개의 비용이다.

질문이 늘면 일회성 준비 비용은 여러 질문에 나누어 볼 수 있다. 예를 들어 질의당 상각 입력은 `질의 입력 + 해당 방식의 준비 입력 / 재사용한 질문 수`다. 실제 코드 변경에 따른 재생성 횟수·갱신 비용·비용 회수 시점은 이번 실험에서 측정하지 않았다.

5.6 C의 입력 정리로 줄어든 비용

C는 RH보다 근거 입력을 **20.8%**, 전체 입력을 **18.2%** 줄였다. 같은 세션에서 대표 응답 시간은 **5.79→5.23초**, 변화율은 **9.7% 단축**이었다. 단축률이 음수이면 더 오래 걸렸다는 뜻이다. 속도 변화의 탐색적 95% 구간은 8.3~11.0%다. 독립 검증에서 RH와 C의 **288쌍 모두 선택 자료가 같고, 고유 코드·import·요약 본문이 보존됨을 확인했다.**

문제별 판정에서는 C가 RH보다 개선된 문제가 **2개**, 악화된 문제가 **2개**다. 총정답 수만으로 개별 문제의 결과까지 같다고 판단하면 안 된다.

6. 왜 이런 정답률이 나왔는가

아래 **확인**은 구축 코드와 기존 2,016개 실제 답변 기록에서 직접 확인한 사실이다. **추측**은 그 사실에 근거한 가능한 설명이다. 이 절은 사후 분석이며, 원인을 하나씩 바꾸는 추가 실험을 수행한 결과는 아니다. 정답 수는 반복을 제외한 같은 96문제 기준이다.

6.1 방식별 확인 사실과 가능한 설명

A — 17/96, 17.7%.

확인: 미리 지정한 출발 함수와 선택이 일치한 문제는 68/96개였고, 필요한 원문 평균 포함률은 28.8%였다. 파싱으로 만든 주소록 자체에는 함수의 동작 설명이 없으며, 선택 후에도 같은 파일 안에서만 확장한다.

추측: 출발 함수 선택과 파일 경계 때문에 계산에 필요한 다른 파일의 규칙을 충분히 확보하지 못한 점이 낮은 정답률의 한 원인일 수 있다. 이는 적은 입력량을 얻는 과정에서 생긴 범위 제한으로 해석할 수 있다.

B — 20/96, 20.8%.

확인: A와 선택한 출발 함수가 96/96문제에서 같았지만, 필요한 원문 평균 포함률은 72.4%로 높아졌다. A보다 9문제에서 개선되고 6문제에서 악화됐다. 출발 함수가 지정값과 일치한 68문제 중 23문제는 토큰을 채우기 전부터 필요한 원문을 모두 모으지 못했다. 이 집단에서 모아 둔 필수 원문이 토큰 제한 때문에 빠진 경우는 0개였다.

추측: 파일 간 원문 보강이 일부 문제에 도움이 됐지만, 2차례의 정적 확장만으로 깊은 참조나 별도 경로를 모두 확보하기 어려웠을 수 있다. 입력 범위와 표시 형식도 달라지므로 순수한 확장 효과의 크기를 이 결과만으로 분리하지는 않는다.

R0 — 25/96, 26.0%.

확인: 필요한 원문 평균 포함률은 99.0%, 지정 원문이 모두 들어간 문제는 94/96개였다.

추측: 함수 하나의 선택에 묶이지 않고 여러 파일의 원문을 찾으며, 자연어로 바꾸는 과정 없이 수식·상수·조건을 전달한 점이 이번 표본에서 유리했을 수 있다. 다만 원문이 모두 있는 94문제 중 69문제도 틀렸다. 검색 이후의 코드 해석·계산·정확한 출력 생성 역시 병목 후보다.

RF — 15/96, 15.6%.

확인: R0보다 10문제에서 악화됐고 새로 맞힌 문제는 0개였다. 필요한 원문 평균 포함률은 53.1%로 내려갔다. 함수 요약 선택 1,443건 중 897건(62.2%)은 파일을 늘리려고 추가한 보조 함수의 요약이었다. 이 보조 함수들은 평가 문제의 필수 원문 목록에 없다.

추측: 같은 1,024토큰 안에 요약이 들어오면서 세부 원문이 밀려나고, 질문과 표현만 비슷한 설명이 공간을 차지했을 가능성이 있다. 짧게 요약하는 과정에서 수치·분기 조건이 빠지는 문제도 가능한 원인이다. 다만 모든 오답을 요약의 정보 손실 하나로 설명할 수는 없다.

RH — 16/96, 16.7%.

확인: RF보다 새로 맞힌 문제는 1개였다. 파일 요약이 선택된 문제는 8/96개, 프로젝트 요약이 선택된 문제는 0/96개였다.

추측: 계층을 만들어도 상위 요약이 답변에 거의 사용되지 않아, 그 이점이 성능으로 연결될 기회가 적었던 것으로 보인다. 유일하게 개선된 캐시 키 문제(`eval_cache__canonical_key`)의 파일 요약도 주로 보조 함수들을 설명했다. 이 한 사례를 '상위 요약이 정확한 규칙을 새로 제공해서 개선됐다'는 증거로 해석하기는 어렵다. 요약 선택과 입력 구성의 변화가 출력을 바꿨을 가능성도 있다.

C — 16/96, 16.7%.

확인: RH와 선택 자료 및 고유 본문은 같고, 총 입력은 18.2% 줄었다. 정답은 2문제 개선·2문제 악화로 총수가 같았다.

추측: 중복·제목·배치의 변화가 모델이 자료를 읽고 답을 구성하는 데 영향을 줬을 수 있다. 본문 보존은 확인했지만, 모델 출력까지 보존되는 것은 아니다. 새 요약 정보를 더하지 않았으므로 정확도 개선이 자동으로 생길 이유도 없다.

파일 전체 — 15/96, 15.6%.

확인: 필요한 원문 평균 포함률은 30.6%이며, 지정 원문이 모두 들어간 문제는 14/96개다. 다른 파일의 정의는 따라가지 않고 관계없는 보조 함수도 모두 전달한다. 입력 잘림은 없었으며, 7문제는 세 반복 모두 답변 출력 한도에서 끝났다.

추측: 한 파일을 완전히 읽어도 파일 밖의 필수 규칙은 빠지고, 파일 안에서는 필요한 규칙을 많은 보조 코드 사이에서 구분해야 하는 점이 불리했을 수 있다. 출력 한도 종료도 측정된 오답의 일부다.

6.2 실제로 선택된 원문과 요약

실제로 어떤 종류의 근거를 골랐는지도 확인했다. '선택 수'는 288번의 호출에서 선택한 항목 수 합계로, 고유한 요약 문서 수와 다르다.

방식	함수 요약 선택 수	상위 요약 선택 수	요약이 들어간 문제 / 96	상위 요약이 들어간 문제 / 96	지정 원문 평균 포함률
원문 검색(R0)	0	0	0	0	99.0%
함수 요약(RF)	1443	0	91	0	53.1%
계층 요약(RH)	1419	27	91	8	53.3%
계층 요약 정리(C)	1419	27	91	8	53.3%

원문 포함률은 필요한 원문 코드가 실제 입력에 들어간 비율이다. 요약의 출처 목록에 함수가 들어 있다는 이유만으로 그 함수의 내용을 모두 전달했다고 계산하지 않았다. 요약이 맞는 내용을 전달할 수도 있으므로, 원문 포함률만으로 요약 방식의 답변 품질을 판정할 수도 없다.

6.3 검색 가중치에서 발견한 단서와 반례

RF·RH의 검색에서 추가로 발견한 단서. 평가용 6개 프로젝트 모두에서 원문 후보에는 단어 `function`이 없었지만, 함수 요약에는 전부 들어 있었다. 현재 구현은 단어 희소성을 원문 후보로만 계산하므로 이 단어를 요약에서도 희귀한 단어처럼 가중한다. `function`을 포함한 질문은 24개였다. **추측:** 이런 질문에서는 구체적 코드 관계보다 반복되는 설명 용어가 검색 점수에 영향을 줬을 수 있다. 전체 후보로 희소성을 다시 계산하는 비교 실험은 하지 않았으므로, 실제 하락 중 얼마가 이 요인 때문인지는 아직 모른다.

원문 누락만으로 설명되지 않는 실제 사례도 있다. `can_edit_own('u1',['document:write'],100,3700)` 문제에서는 R0와 RF 모두 지정 원문을 전부 전달했지만, 정답 `false`에 대해 R0는 `false`, RF는 `true`라고 답했다. 따라서 '원문을 더 찾으면 모든 오류가 해결된다'거나 '요약이 틀려서 모든 오류가 생겼다'고 단정할 수 없다. 특히 R0의 오답 71개 중 67개는 정답 값 불일치, 4개는 JSON 형식 오류였다. 낮은 점수를 출력 형식 문제만으로 설명할 수도 없다.

코드와 기록을 대조해 재계산한 근거는 생성 과정·오답 사후 분석에 보존했다. 기존 정답률이나 평가 기준은 바꾸지 않았다.

7. 목적에 따른 판단과 한계

7.1 무엇을 우선하느냐에 따라 판단이 달라진다

우선하는 목적	이번 실험에서 확인한 판단
한 번에 쓰는 컨텍스트 최소화	**A**의 한 호출 입력이 가장 작았다. 큰 파일에서는 평균 342.0토큰으로 파일 전체 4265.8토큰보다 92.0% 적었다. 작은 파일에서는 코드 자료 42.4%, 가장 큰 호출 입력 20.2%를 줄였지만 누적 입력은 29.2% 늘었다.
이번 문제에서 더 많이 맞히기	**R0**가 작은 파일 27.1%, 큰 파일 26.0%로 관측 정답률이 가장 높았다. 하지만 이 점수로 충분한 답변 신뢰성을 확보했다고 보기는 어렵다.
RH와 같은 자료를 더 짧게 전달하기	**C**는 고유 본문을 보존하면서 RH보다 총 입력을 18.2% 줄였다. 총정답 수는 같아도 2문제 개선·2문제 악화가 있어 개별 답변까지 같지는 않았다.

큰 파일에서는 A·B·R0 모두 파일 전체보다 총 입력을 줄였다. 작은 파일에서는 출발 함수를 고르는 A·B의 추가 호출 비용이 상대적으로 크게 작용했다. **적은 토큰, 넓은 원문 범위, 많은 검색 후보 중 어느 하나만으로 정답이 보장되지는 않는다.** 실제 정답률·입력 범위·호출 수·시간·불확실성을 함께 판단해야 한다.

7.2 이 결과로 말할 수 있는 범위

평가는 **영어 질문, 구조가 비슷한 합성 Python 프로젝트 6개, 한 모델·한 장비**에 한정된다. 파일 전체 방식에는 대상 파일 경로를 미리 알려줬고, 방식마다 실제로 전달한 정보의 범위도 다르다. 전체 구현의 비교이며, 각 요소의 효과를 분리한 비교는 아니다.

큰 파일은 원래의 짧은 질문 대상 함수를 유지하고 관계없는 보조 함수를 추가해 만들었다. 파일 크기와 함께 검색 후보 수와 줄 위치도 바뀌었다. 따라서 **관련 함수 자체가 수천 줄인 경우, 복잡한 실제 저장소와 import, 다른 언어·한국어 질문·보안 취약점 탐지**에서 같은 절감률이나 정확도, 우위를 보장하지 않는다.

프로젝트 수가 적어 통계는 탐색적이다. 시간 비교에도 세션·장비 부하·열 상태의 한계가 있다. 관측값은 유용한 단서지만, 보정 검정에서 차이를 확정하지 못한 결과를 성능 동등성의 증거로 읽어서는 안 된다.

7.3 원인을 확인하기 위한 후속 실험

원인을 더 분리하려면 **정답에 필요한 원문을 고정한 채 요약만 추가**, **원문 전용 가중치와 전체 후보 가중치 비교**, **같은 RH 선택에서 중복 제거만 적용하거나 제목만 축약**하는 실험이 필요하다. 이는 후속 검증 제안이며 이번에 실행한 실험 목록에 포함하지 않았다. 구축 방식과 정답률에 관한 위 해석은 이번 합성 코드·모델·질문·길이 제한에 대한 가설로 읽어야 한다.

부록 A. 상세 성적과 통계

A.1 큰 파일 네 방식의 상세 측정값

4.1절의 일곱 방식 비교에 합쳐진 **기존 A·B·R0·파일 전체 실행의 상세표**다. 별도의 추가 실험을 뜻하지 않는다.

지표	현재 색인(A)	파일 간 확장(B)	원문 검색(R0)	파일 하나 전체
정답 수 / 96문제	17	20	25	15
정답률	17.7%	20.8%	26.0%	15.6%
대표 응답 시간	2.62초	3.50초	5.00초	14.35초
느린 요청의 시간: p95	4.89초	5.41초	5.90초	18.69초
코드 자료 평균	97.2토큰	366.0토큰	1016.1토큰	4117.6토큰
총 입력 평균: 선택 + 답변	500.9토큰	769.4토큰	1164.7토큰	4265.8토큰
가장 큰 한 호출 입력의 평균	342.0토큰	591.1토큰	1164.7토큰	4265.8토큰
8,192토큰 중 입력 점유	4.17%	7.22%	14.22%	52.07%
지정 원문 근거의 평균 포함률	28.8%	72.4%	99.0%	30.6%
지정 원문 근거를 모두 포함한 비율	14.6%	49.0%	97.9%	14.6%
질문당 모델 호출 수	2회	2회	1회	1회
실제 평가 답변 수	288회	288회	288회	288회
답변 출력 한도로 종료한 횟수	0회	0회	0회	21회

문제·방식별 답은 세 반복 모두 같았으므로 정답 수는 서로 다른 96문제를 기준으로 표시했다. 대표 시간은 각 문제의 세 번 중 앙값을 구한 뒤 평균했다. 코드 자료에는 원문과 붙여 보낸 제목·import문·코드 표시가 포함된다. 총 입력은 모든 호출의 질문·지시문·대화 형식까지 포함한다. 출력 토큰은 이 표의 입력량에 포함하지 않는다.

출력 제한: 파일 전체 방식에서 7문제의 세 반복, 총 21회가 공통 답변 한도 128토큰에서 끝났다. 이 응답들도 처음부터 오답에 포함했으며 제외하거나 재시도해 대체하지 않았다. 해당 출력은 정답 목록에 없는 보조 함수 이름을 나열하다가 끝났다. 입력 원문이 잘린 경우는 없었고, A·B·R0에는 출력 한도 종료가 없었다. 보고된 정답률과 시간은 이 공통 출력 한도에서 얻은 결과다.

A.2 작은 파일: 질문 종류·프로젝트·문제 집합별 정답

각 칸은 서로 다른 문제의 정답 수다.

질문 종류	문제 수	A	B	R0	파일 전체
한 함수의 계산	12	8	8	8	8
여러 파일을 잇는 계산	24	1	1	2	0
경계 조건	12	4	3	4	5
자연어로 설명한 동작	24	2	3	4	1
설정 변경의 영향	12	0	0	4	2
함수 호출 구조	12	2	7	4	1

프로젝트 주제	A	B	R0	파일 전체
가격	2 / 16	3 / 16	4 / 16	2 / 16
권한	5 / 16	5 / 16	8 / 16	3 / 16
라우팅	4 / 16	7 / 16	6 / 16	5 / 16
재고	2 / 16	4 / 16	5 / 16	3 / 16
캐시	2 / 16	1 / 16	1 / 16	2 / 16
일정	2 / 16	2 / 16	2 / 16	2 / 16

문제 집합	A	B	R0	파일 전체
기존 48문제	8 / 48	12 / 48	12 / 48	6 / 48
추가 48문제	9 / 48	10 / 48	14 / 48	11 / 48

질문 종류별 문제 수가 작고 두 문제 집합이 같은 프로젝트를 공유한다. 이 표는 차이가 어디에 모였는지 보여 주는 보조 분석이다.

A.3 큰 파일: 질문 종류별 정답

A·B·R0·파일 전체 — 각 칸은 해당 종류에서 맞힌 문제 수다.

질문 종류	문제 수	A	B	R0	파일 전체
한 함수의 계산	12	8	7	7	8
여러 파일을 잇는 계산	24	1	0	2	0
경계 조건	12	4	3	4	3
자연어로 설명한 동작	24	2	2	4	1
설정 변경의 영향	12	0	1	4	1
함수 호출 구조	12	2	7	4	2

R0·RF·RH·C

질문 종류별 정답 수는 다음과 같다. 종류별 표본이 작으므로, 이 표만으로 특정 종류에 대한 우위를 일반화하지 않는다.

질문 종류	문제 수	R0	RF	RH	C
한 함수의 계산	12	7	7	7	8
여러 파일을 잇는 계산	24	2	2	3	3
경계 조건	12	4	2	2	1
자연어로 설명한 동작	24	4	2	2	2
설정 변경의 영향	12	4	1	1	1
함수 호출 구조	12	4	1	1	1

A.4 작은 파일: 파일 전체와의 통계 비교

이번에는 **파일 전체 – 비교 방식**으로 방향을 맞췄다. %p는 두 정답률의 단순 차이다.

비교	정답률 차이	탐색적 95% 추정 구간	파일 전체 승 / 패 / 동률	보정 p값
파일 전체 – A	+0.0%p	-5.2 ~ +4.2%p	4 / 4 / 88	1.000
파일 전체 – B	-5.2%p	-10.4 ~ +0.0%p	6 / 11 / 79	0.562
파일 전체 – R0	-9.4%p	-18.8 ~ -1.0%p	3 / 12 / 81	0.562

6개 프로젝트를 묶음으로 다시 뽑는 46,656가지 조합으로 추정 구간을 계산했다. 1,152답변을 독립된 1,152문제로 취급하지 않았다. 새 세 비교의 프로젝트 단위 검정에는 Holm 보정을 적용했다. 구간이 0을 포함하거나 보정 p값이 0.05보다 크면 해당 방법으로 우위를 확정하지 않는다. 프로젝트 수가 적어 이 통계는 탐색적이며, 작은 합성 자료 밖으로 일반화하는 근거는 부족하다.

특히 R0은 파일 전체보다 9문제를 더 맞혔지만, 보정 p값은 0.562였다. 표의 추정 구간과 p값은 서로 다른 계산법에서 나온 값이며, 이 결과만으로 R0의 일반적인 우위를 입증했다고 해석하지 않는다.

A.5 큰 파일: 네 방식 사이의 통계 비교

비교: 앞 방식 – 뒤 방식	정답률 차이	탐색적 95% 추정 구간	보정 p값
파일 간 확장(B) – 현재 색인(A)	+3.1%p	-4.2 ~ +10.4%p	1.000
원문 검색(R0) – 현재 색인(A)	+8.3%p	+1.0 ~ +15.6%p	1.000
원문 검색(R0) – 파일 간 확장(B)	+5.2%p	-1.0 ~ +12.5%p	1.000
현재 색인(A) – 파일 하나 전체	+2.1%p	-2.1 ~ +7.3%p	1.000
파일 간 확장(B) – 파일 하나 전체	+5.2%p	-4.2 ~ +16.7%p	1.000
원문 검색(R0) – 파일 하나 전체	+10.4%p	+1.0 ~ +19.8%p	1.000

6개 프로젝트를 묶음으로 재표집하는 46,656가지 조합으로 추정 구간을 계산했다. 3회 반복을 독립된 문제로 세지 않았으며, 프로젝트 단위 부호 뒤집기 검정의 p값에 여섯 비교에 대한 Holm 보정을 적용했다. 추정 구간과 p값은 서로 다른 방법으로 계산한 값이다. 따라서 구간이 0을 제외하더라도 보정 p값이 0.05보다 크면 검정상 우위를 확정하지 않는다. 프로젝트가 6개뿐이므로 결과는 탐색적이다.

A.6 큰 파일: 함수 요약·계층 요약·입력 정리의 통계 비교

비교	정답률 차이	탐색적 95% 구간	보정 p값	개선 / 악화 / 동일 문제
함수 요약(RF) – 원문 검색(R0)	-10.4%p	-12.5 ~ -8.3%p	0.094	0 / 10 / 86
계층 요약(RH) – 함수 요약(RF)	+1.0%p	+0.0 ~ +3.1%p	1.000	1 / 0 / 95
계층 요약 정리(C) – 계층 요약(RH)	+0.0%p	-5.2 ~ +4.2%p	1.000	2 / 2 / 92

이번 추가 실험에서 미리 지정한 세 비교에 대해서만 Holm 보정을 적용했다. 6개 프로젝트 단위의 46,656가지 재표집으로 95% 구간을, 프로젝트별 부호 뒤집기로 p값을 계산했다. 두 계산법이 달라 구간과 검정 결과를 동일시하지 않는다. 같은 문제의 세 반복은 독립된 세 문제가 아니며, 이 작은 합성 표본은 탐색적 근거다.

검정의 제약: 프로젝트가 6개이면 이 양측 부호 검정의 최소 p값은 2/64이고, 세 비교를 Holm 보정한 최소값은 0.09375다. 따라서 이 설계만으로 0.05 기준의 우위를 확정할 수 없다. 보정 p값이 크다는 사실을 '효과가 없다'거나 '성능이 같다'는 증거로 읽지 않고, 관측 차이와 추정 구간을 함께 본다.

부록 B. 같은 질문의 실제 답변

재고·권한의 기존 두 사례와 RF·RH·C 비교 사례를 모았다. 전체 성능을 대표하도록 뽑은 사례는 아니다.

B.1 작은 파일의 재고 계산

질문: `What does remaining_after(9,1,8,True) return?`
정답: `0`

항목	A	B	R0	파일 전체
실제 답변	`8`	`0`	`3`	`8`
코드 자료 토큰	182	525	1015	197
정답 여부	오답	정답	오답	오답

재고 9개에서 이미 잡힌 1개와 안전재고 3개를 빼면 판매 가능량은 5개다. 8개를 요청해도 부분 예약은 5개만 되므로 남는 수량은 0개다. 파일 전체 방식에서는 `service.py` 전체를 보냈다. 다른 파일에 정의된 함수·상수가 자동으로 따라오는 것은 아니다. 이 표는 첫 번째 반복의 실제 출력이며, 실패 원인의 기여도를 분리한 실험은 아니다. 기록 식별자: `eval_inventory__v2_partial_depletes_available`.

B.2 같은 재고 문제를 큰 파일에서 물으면

부록 B.1과 같은 재고 문제다. `remaining_after(9,1,8,True)`의 정답은 `0`이며, 원래 계산 코드는 바꾸지 않았다.

항목	A	B	R0	파일 전체
실제 답변	`8`	`5`	`3`	`8`
코드 자료 토큰	182	562	1021	4103

파일 전체 방식은 커진 `service.py`의 모든 줄을 입력했다. 다른 파일에 있는 계산 함수·정책 상수는 그 파일 안에 자동으로 들어오지 않는다. 각 값은 첫 번째 반복의 실제 출력이다. 이전부터 사용한 사례를 그대로 비교했으며, 전체 결과를 대표하도록 골라낸 사례는 아니다.

B.3 작은 파일의 권한 계산

질문: `What does can_edit_own('k8',[' DOCUMENT:WRITE ','document:write'],50,1849) return?`
정답: `true`

항목	A	B	R0	파일 전체
실제 답변	`true`	`false`	`true`	`false`
코드 자료 토큰	158	447	1022	161
정답 여부	정답	오답	정답	오답

공백과 대문자를 정리하면 유효한 쓰기 권한이 남는다. 세션 경과 시간도 1,799초로 제한 3,600초보다 짧으므로 true다. 파일 전체 방식에서는 `service.py` 전체를 보냈다. 다른 파일에 정의된 함수·상수가 자동으로 따라오는 것은 아니다. 이 표는 첫 번째 반복의 실제 출력이며, 실패 원인의 기여도를 분리한 실험은 아니다. 기록 식별자: `eval_authorization__v2_duplicate_write_permission`.

B.4 큰 파일: 요약 추가·입력 정리 후 답변이 달라진 사례

표는 첫 번째 반복의 답변에서 읽어낸 판정용 JSON값이다. JSON으로 읽을 수 없는 응답은 'JSON 형식 오류'로 표시했다. 응답 원문과 판정 실패 사유는 평가 원자료에 함께 보존했다.

요약을 추가해 틀린 사례 — `eval_authorization__direct_calls`

List the project-defined functions directly called by authorize. Ignore builtins, and return a sorted list of function names.

항목	판정용 답변
정답	["role_allows", "scope_allows", "session_valid"]
원문 검색(R0)	["role_allows", "scope_allows", "session_valid"]
함수 요약(RF)	[]
계층 요약(RH)	[]
계층 요약 정리(C)	["aux_audit_006_moving_average", "aux_audit_018_moving_average", "aux_audit_042_moving_average", "aux_audit_054_moving_average", "aux_presentation_006_moving_average", "aux_presentation_018_moving_average", "aux_presentation_030_moving_average"]

정리 후 판정이 달라진 사례 — `eval_authorization__own_write`

What does authorize('u1','u1','member',[' DOCUMENT:WRITE '],'write',100,200,False) return?

항목	판정용 답변
정답	{"allowed": true, "reason": "allowed"}
원문 검색(R0)	{"allowed": true, "reason": "allowed"}
함수 요약(RF)	{"allowed": true, "reason": "allowed"}
계층 요약(RH)	{"allowed": true, "reason": "allowed"}
계층 요약 정리(C)	JSON 형식 오류

위 사례는 판정이 바뀐 질문 중 ID 순서로 처음 나타난 것을 골라 보여 준 것이다. 어느 문장이나 요약 오류가 답변 변화를 일으켰는지는 따로 제거 실험을 하지 않았으므로 원인으로 단정하지 않는다.

부록 C. 원자료·검증·재현

아래 링크는 기존 실행 기록과 검증 자료다. **이번 편집은 문서의 배치·제목·표 구성을 정리한 것이며, 새 LLM 실험이나 정답률 변경은 없다.**

C.1 작은 파일 실험

파일 전체 288회가 실제 생성된 요청·응답인지, 입력이 파일 원문과 정확히 같은지, 캐시·시간·토큰·정답·집계가 일치하는지 **20개 점검을 통과**했다. A·B·R0의 기존 수치는 그대로 유지했다. 로컬 기록 검증이며 외부 인증은 아니다.

확인할 자료	링크
네 방식의 통합 결과·질문별 답변	통합 집계
파일 전체 실제 288회 실행	실행 기록 · 실제 모델 호출 로그
계획·정답·환경·검증	실험 계획 · 정답 재확인 · 환경 · 검증
1절 예제의 실제 파일과 생성된 색인	service.py · rules.py · policy.py · 생성 자료
보존한 기존 실행	A·B 576회 · R0 288회
재현 안내	실행·검증·보고서 생성 방법
이번 수정 전 보고서	보관본

C.2 큰 파일의 A·B·R0·파일 전체 실험

새 1,152회 답변과 실제 요청·응답, 원문 전체 입력, 정답, 토큰, 집계에 대해 **22개 점검을 통과**했다. 측정 기록을 확인하는 로컬 검증이며 외부 인증은 아니다.

확인할 자료	링크
큰 파일 통합 결과·질문별 답변	통합 집계
실제 평가 실행·모델 요청	1,152회 실행 기록 · 모델 호출 로그
고정한 계획·파일 구성	실험 계획 · 파일 크기와 확장 내역
환경·정답·검증	실행 환경 · 정답 재확인 · 검증 결과 · 출력 한도 종료 내역
실제 큰 파일 예시	재고 service.py · 재고 rules.py
재현	실행·분석·검증 안내
추가 전 문서	작은 파일 보고서 보관본

C.3 큰 파일의 RF·RH·C 추가 실험

확인할 내용	기록
실행 전 고정 조건	실험 계획
생성한 요약·출처·계층	요약 자료
새 864개 답변과 실제 입력	평가 원자료
사전 요약부터 답변까지 실제 모델 요청	호출 기록

확인할 내용	기록
일곱 방식 집계·같은 48문제 비교	집계
독립 재계산·토큰·검색·본문 보존 점검	검증
기존 보고서 보존본	수정 전 보고서

C.4 문서 재구성 전후 확인

정리 전 보고서 · 내용 이동·통합 대조표 · 누락·수치·링크 확인

C.5 독자 관점의 설명 점검

맥락 공백과 보완 내용 · 기존 내용·수치 보존 확인. 실제 독자 대상 이해도 시험이 아닌 문서의 설명 흐름과 근거 연결에 대한 편집 검토다.